

Algorithms & Probability Recap

A complete recap of the contents taught in the Algorithms and Probability lectures at ETH Zürich, which are part of the Basisjahr's second semester.

Erik Girdauskas

Table of Contents

Weeks 1-2: Connectivity and Cycles	1
Menger and Blocks	1
Modified DFS, Cut Edges	2
Cycles	3
TSP	4
Weeks 3-4: Matchings and Colorings	5
Hopcroft-Karp	6
K-Coloring	8
Weeks 5-6: Early Probability Theory	11
Conditional Probability	12
Random Variables	14
Stable Set Example	16
Weeks 7-8: Advanced Probability Theory	17
Important Distributions	17
Important Inequalities	20
Coupon Collector	21
Chernoff Bounds	22
Weeks 9-10: Randomized Algorithms	23
Las Vegas vs. Monte Carlo	23
Prime Number Testing	25
Quicksort	26
Quickselect	27
Hashing	28
Weeks 11-12: General Algorithm Highlights	29
Long Path Problem	29
Flow Networks	31
Bipartite Matching via Flow	33
Weeks 13-14: More General Algorithms	35
Karger's Random Contraction Algorithm	35
Smallest Enclosing Disk	36
Convex Hull	38
Additional Exam Practice	41
Coupon Collector and Matching	41
Smallest Enclosing Disk	42

AUW semester recap

Weeks 1-2:

k-connectivity: (considering a graph $G = (E, V)$)

→ u and v are connected if $\exists u-v$ -path in G

→ G is connected if for all $u, v \in V$ with $u \neq v$ $\exists u-v$ -path in G

→ u and v are **k-connected** if $|V| \geq k+1$ and $\forall X \subseteq V \setminus \{u, v\}$ with $|X| < k$ is $u-v$ -connected in $G[V \setminus X]$

→ G is **k-connected** if $|V| \geq k+1$ and for all $X \subseteq V$ with $|X| < k$, $G[V \setminus X]$ is connected

→ X is an **u-v separator** if u and v are in separate connected components of $G[V \setminus X]$

→ G is also **k-connected** if $|V| \geq k+1$ and for all $u, v \in V$ every $u-v$ sep. X has $|X| \geq k$.

→ you have to delete at least k vertices to break connectivity

→ If G contains a vertex with degree $< k$, G is not **k-connected**

→ **Menger's Theorem (vertices):** every X has $|X| \geq k \iff \exists k$ vertex disjoint $u-v$ paths

↑ I will generally write X for $u-v$ separators

→ **Menger's Theorem (all vertices):** G is **k-conn.** $\iff$ for all $u, v \in V$ with $u \neq v$, $\exists k$ internally vertex disjoint $u-v$ paths

→ X is a **u-v edge separator** $\Rightarrow u$ and v are in different connected components of $G' = (V, E \setminus X)$

→ G is **k-edge-connected** $\Rightarrow$ for all $u, v \in V$, all X have $|X| \geq k$

→ if all $X \subseteq E$ have $|X| < k$, $(V, E \setminus X)$ is connected (assuming G is **k-edge-connected**)

→ **Menger's Theorem (Edges):** G is **k-edge-connected** $\iff$ for all $u, v \in V$ with $u \neq v$, $\exists k$ edge-disjoint $u-v$ paths

Cut vertices, cut edges: → $v \in V$ is cut vertex if $G[V \setminus \{v\}]$ not conn. and only if $e \in E$ is cut edge if $G \setminus e$ not conn. and only if

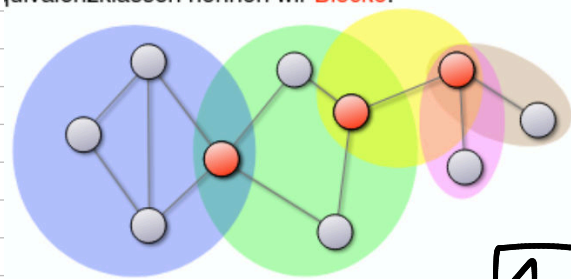
→ $\{x, y\} \in E$ is a cut edge if $\deg(x) = 1$ or $\deg(y) = 1$ or x or y are cut vertices

Blocks: equiv. relation: $e \sim f \iff \begin{cases} e = f \text{ or} \\ \exists \text{ cycle with } e, f \end{cases}$

→ we define the **block graph** as $T = (A \cup B, E_T)$ with

$A = \{\text{cut vertices of } G\}$ and $B = \{\text{blocks of } G\}$

→ $\forall a \in A, \forall b \in B: \{a, b\} \in E_T \iff a$ adjacent to an edge in b



Theorem: if G is connected, T is a tree.

Finding cut vertices using DFS:

We use the following variation from the script:

DFS-VISIT(G, v)

```

1: num  $\leftarrow$  num + 1
2: dfs[v]  $\leftarrow$  num
3: low[v]  $\leftarrow$  dfs[v]
4: isArtVert[v]  $\leftarrow$  FALSE
5: for all  $\{v, w\} \in E$  do
6:   if dfs[w] = 0 then
7:     T  $\leftarrow$  T +  $\{v, w\}$ 
8:     val  $\leftarrow$  DFS-VISIT( $G, w$ )
9:     if val  $\geq$  dfs[v] then
10:      isArtVert[v]  $\leftarrow$  TRUE
11:   low[v]  $\leftarrow$  min[low[v], val]
12: else dfs[w]  $\neq$  0 and  $\{v, w\} \notin T$ 
13:   low[v]  $\leftarrow$  min[low[v], dfs[w]]
14: return low[v]
```

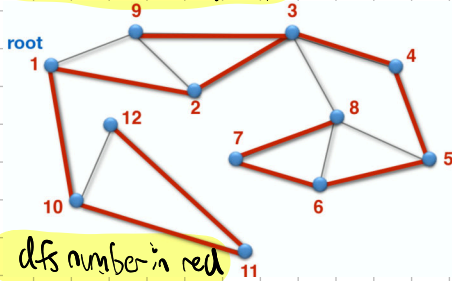
DFS(G, s)

```

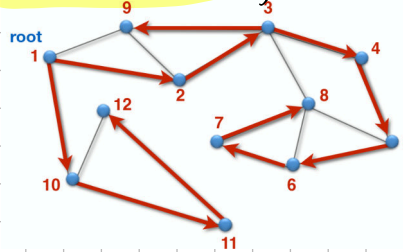
1:  $\forall v \in V: \text{dfs}[v] \leftarrow 0$ 
2: num  $\leftarrow 0$ 
3: T  $\leftarrow \emptyset$ 
4: DFS-VISIT( $G, s$ )
5: if s hat in T Grad mindestens zwei then
6:   isArtVert[s]  $\leftarrow$  TRUE
7: else
8:   isArtVert[s]  $\leftarrow$  FALSE
```

this works as follows:

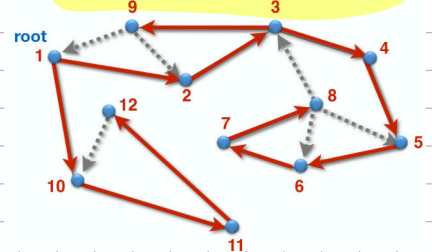
regular DFS graph



directed DFS graph



direct leftover edges away from their Visited vertices



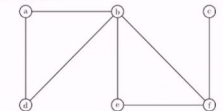
dfs number in red

skills needed to do this on paper:

apply DFS on graph, write down DFS number (when we visit the n th vertex, n is the DFS number)
 $\rightarrow$ direct the edges and leftovers
 $\rightarrow$ start out with $\text{low}[v] = \text{dfs}[v]$
 $\rightarrow$ update for some leftover edges and when the algorithm backtracks

Exercise S2.1 - Efficient Search for Bridges and Cut-Vertices
 In the lecture we have seen how to find bridges (Brücken) and cut-vertices (Artikulationsknoten) in a graph. The algorithm uses a modified DFS and computes values $\text{dfs}[v]$ and $\text{low}[v]$ for every vertex v .

Consider the following graph G :



(a) Perform the modified DFS starting at vertex a . Use the corresponding dfs - and low -values to determine the cut-vertices and bridges of G .

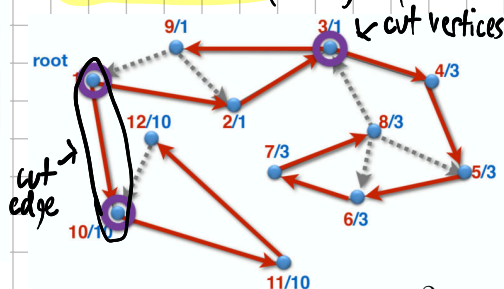
We now define $\text{low}[v] :=$ smallest DFS number reachable through a directed path using an arbitrary amount of tree edges and max. 1 leftover edge

more formally: low values can be computed in $O(|V| + |E|)$

$\text{low}[v] = \min(\text{dfs}[v], \min\{\text{dfs}[w], \text{if } (v, w) \text{ leftover edge}\} \text{low}[w] \text{ if } (v, w) \text{ tree edge})$

$\rightarrow v$ cut vertex $\Leftrightarrow$ 1) $v \neq \text{root}$ and v has child w in DFS tree with $\text{low}[w] \geq \text{dfs}[v]$
 2) $v = \text{root}$ and v has at least 2 children in the DFS tree

in the example graph:



Cut edges: a tree edge $e = (v, w)$ (v parent, w child) is a cut edge if and only if $\text{low}[w] > \text{dfs}[v]$.

Theorem: This modified DFS computes cut vertices and cut edges in a connected graph in $O(m)$

Cycles:

→ Hamilton cycle → cycle in G that contains every vertex exactly once

→ Euler tour → a closed walk in G that contains every edge exactly once

→ Theorem: a connected graph contains an Euler tour if and only if every vertex has an even degree and it can be found in $O(|E|)$

→ the lecture details a "runner and tortoise" algorithm where the runner first finds a closed walk, then the tortoise finds an edge-disjoint closed walk, and so on, until there are no edges (success, Euler tour) or there are no closed walks (no Euler tour)

→ algo. from script:

EULERTOUR(G, v_{start})

```

1: // Schneller Läufer
2:  $W \leftarrow \text{RANDOMTOUR}_G(v_{start})$ 
3: // Langsamer Läufer
4:  $v_{\text{langsam}} \leftarrow \text{Startknoten von } W$ 
5: while  $v_{\text{langsam}}$  ist nicht letzter Knoten in  $W$  do
6:    $v \leftarrow \text{Nachfolger von } v_{\text{langsam}} \text{ in } W$ 
7:   if  $N_G(v) \neq \emptyset$  then
8:      $W' \leftarrow \text{RANDOMTOUR}_G(v)$ 
9:     // Ergänze  $W = W_1 + \langle v \rangle + W_2$  um die
10:    // Tour  $W' = \langle v'_1 = v, v'_2, \dots, v'_{l-1}, v'_l = v \rangle$ 
11:     $W \leftarrow W_1 + W' + W_2$ 
12:    $v_{\text{langsam}} \leftarrow \text{Nachfolger von } v_{\text{langsam}} \text{ in } W$ 
13: return  $W$ 
```

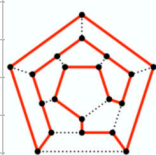
RANDOMTOUR $_G(v_{start})$

```

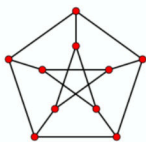
1:  $v \leftarrow v_{start}$ 
2:  $W \leftarrow \langle v \rangle$ 
3: while  $N_G(v) \neq \emptyset$  do
4:   Wähle  $v_{\text{next}}$  aus  $N_G(v)$  beliebig.
5:   Hänge  $v_{\text{next}}$  an die Tour  $W$  an.
6:    $e \leftarrow \{v, v_{\text{next}}\}$ 
7:   Lösche  $e$  aus  $G$ .
8:    $v \leftarrow v_{\text{next}}$ 
9: return  $W$ 
```

Hamilton cycles:

use these as counterexamples in proofs:



Ikosaeder

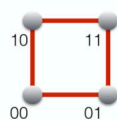


Petersengraph

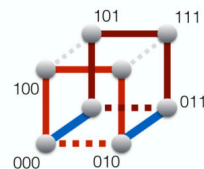
Theorem: an $n \times m$ grid contains a Hamilton cycle if and only if $n \cdot m$ is even.

Hypercubes and gray codes:

$d=2$:



$d=3$:



add extra bit,
then put
reversed
list below
the normal
one

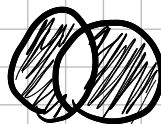
analog für $d \geq 4$

Theorem (Dirac): If $|V| \geq 3$ and min. degree $\delta(G) \geq |V|/2$, G contains a Hamilton cycle.

Sieve formula (Inclusion/Exclusion): for finite sets $A_1, \dots, A_n$ ($n \geq 2$):

$$|\bigcup_{i=1}^n A_i| = \sum_{i=1}^n (-1)^{i+1} \sum_{1 \leq i_1 < \dots < i_i \leq n} |A_{i_1} \cap \dots \cap A_{i_i}| = \sum_{i=1}^n |A_i| - \sum_{1 \leq i_1 < i_2 \leq n} |A_{i_1} \cap A_{i_2}| + \sum_{1 \leq i_1 < i_2 < i_3 \leq n} |A_{i_1} \cap A_{i_2} \cap A_{i_3}| - \dots + (-1)^{n+1} \cdot |A_1 \cap \dots \cap A_n|$$

for $n=2$: $|A_1 \cup A_2| = |A_1| + |A_2| - |A_1 \cap A_2|$



Theorem: One can decide and, if applicable, find a Hamilton cycle in $O(|V|^2 \cdot 2^{|V|})$
(if a Hamilton cycle exists)

Theorem (Karp): Deciding if a graph contains a Hamilton cycle is NP-complete

Hamilton algorithm (DP): Time complexity: $n^2 \cdot 2^n$, Space: $n \cdot 2^n$

Idea: For all $S \subseteq [n]$ with $1 \in S$ and all $x \in S$ with $x \neq 1$:

$$P_{S,x} := \begin{cases} 1, & \exists 1-x\text{-path containing exactly the vertices from } S \\ 0 & \text{else} \end{cases}$$

Initialization: $\forall x \in \{2, \dots, n\}$: for $S = \{1, x\}$ set $P_{S,x} = 1$ if and only if $\{1, x\} \in E$.

Loop: for all $s = 3$ to n
for all $S \subseteq [n]$ with $1 \in S$ and $|S| = s$:
for all $x \in S$ with $x \neq 1$:

$$P_{S,x} = \max \{ P_{S \setminus \{x\}, y} : y \in N(x) \cap S, y \neq 1 \}$$

Output: G contains Hamilton cycle $\Leftrightarrow \exists x \in N(1)$ with $P_{[n],x} = 1$

Traveling Salesman: given a complete graph with n vertices and the distance $l: \binom{[n]}{2} \rightarrow \mathbb{R}$ between any 2 vertices, find the smallest round trip:

$\min_{\text{Hamilton cycle}} \sum_{e \in E(H)} l(e) \rightarrow$ find a round trip smaller than a given integer X .

It is NP-complete: Graph $G = ([n], E) \Leftrightarrow$ complete graph $K_n = ([n], \binom{[n]}{2})$
with length function $l: \binom{[n]}{2} \rightarrow \{0, 1\}$
s.t. $l(\{x, y\}) = \begin{cases} 0, & \text{if } \{x, y\} \in E \\ 1 & \text{else} \end{cases}$

Then: G includes Hamilton cycle $\Leftrightarrow \text{opt}(K_n, l) = 0$

From this: there cannot be a polynomial C -approx. algo. for any $C > 0$

we can derive the metric TSP if:

l fulfills triangle inequality: $l(x, z) \leq l(x, y) + l(y, z) \quad \forall x, y, z \in [n]$

$\rightarrow$ we can derive a 2-approximation using the MST

Algo.: 1. find MST T ($l(T) \leq \text{opt}(K_n, l)$)

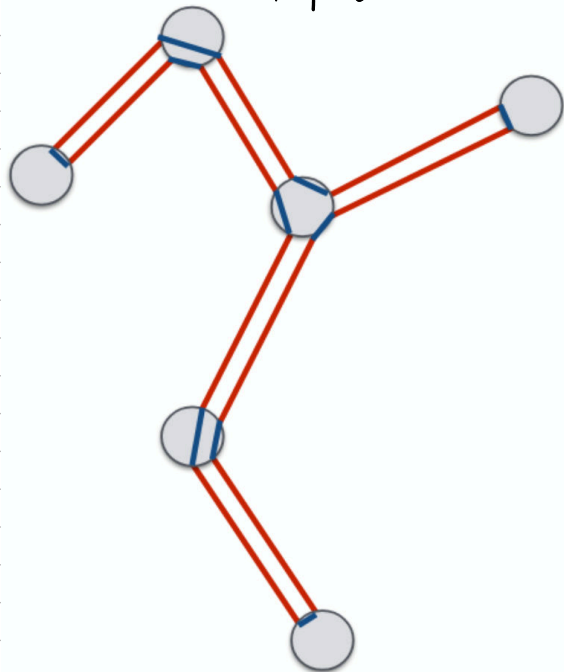
2. double all edges of T ($2l(T) \leq 2\text{opt}(K_n, l)$)

3. find Euler tour W ($l(W) = 2l(T) \leq 2\text{opt}(K_n, l)$)

4. go through W , use any shortcuts s.t. any vertex is visited only once

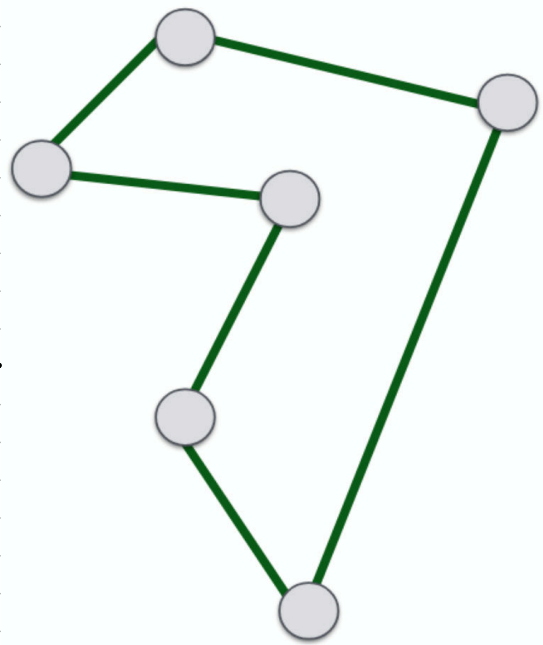
$\rightarrow$ Hamilton cycle C , $l(C) \leq l(W) = 2l(T) \leq 2\text{opt}(K_n, l)$

After step 3:



Step 4

this is allowed
because we
have a
complete graph



Weeks 3 and 4:

Matchings: $M \subseteq E$ is a matching if no vertex is adjacent to more than one edge of M .

$\rightarrow M$ covers v if $\exists e \in M$ that contains v

$\rightarrow$ perfect matching: $|M| = |V|/2$ (every vertex is covered)

$\rightarrow$ maximal matching: $\nexists M'$ with $M \subsetneq M'$ and $|M'| > |M|$

$\rightarrow$ maximum matching: $\exists M'$ with $|M'| > |M|$ (stronger condition)

Theorem: with the greedy algorithm, we can find a maximal matching M_{greedy} with $|M_{\text{greedy}}| \geq |M_{\text{max}}|/2$ (M_{max} is maximum)

GREEDY-MATCHING (G)

- 1: $M \leftarrow \emptyset$
- 2: while $E \neq \emptyset$ do
- 3: wähle eine beliebige Kante $e \in E$
- 4: $M \leftarrow M \cup \{e\}$
- 5: lösche e und alle inzidenten Kanten in G

Observations: $\rightarrow$

$\rightarrow$ for any edge in M_{max} , at least one of two vertices adjacent is covered by an edge from M_{greedy}

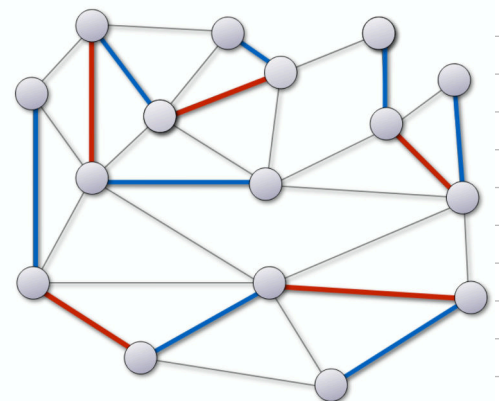
$\rightarrow$ any edge from M_{greedy} can at most cover 2 edges from M_{max}

augmenting paths: an M -augmenting path alternates between edges in M and not in M and starts/ends with edges not in M .

$\rightarrow$ we can increase M 's size by swapping along $M \Rightarrow M' := M \oplus P$

M_{greedy}

M_{max}

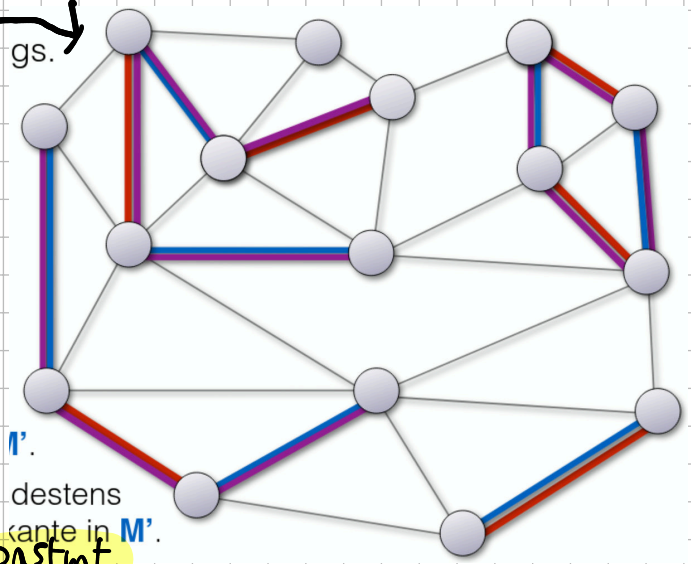


Beobachtung:

XOR

what happens if you have $M \oplus M'$ with M and M' being arbitrary matchings?

Observations:



- every vertex has degree ≤ 2
- $\Rightarrow$ collection of paths and cycles
- every path/cycle alternates between edges from M, M'
- if $|M| < |M'|$, $\exists$ at least one path with start and ending edge in M'

Finding an augmenting path: $O(|V| \cdot |E|)$ constant
 state of the art matching: $O(|E|^{1+o(1)})$ for bipartite graphs/
 $O(|V|^{1/2} \cdot |E|)$ in general

Theorem (Berge): Every non-maximum matching has an augmenting path

Algorithm: (outputs maximum matching M)

Start with $M = \emptyset$.

- repeat
- look for aug. path P $\nwarrow$ $\begin{matrix} \text{bipartite} \\ \text{In bip. graphs: } O(|V| \cdot |E|) \text{ (with BFS)} \\ \text{in general: } O(|V| \cdot |E|) \text{ (Edmond's blossom algorithm)} \end{matrix}$
 - if it doesn't exist, return M
 - else $M := M \oplus P$

Theorem (Hall):

A bip. graph $G = (A \oplus B, E)$ contains matching M with $|M| = |A|$ if and only if
 $\forall X \subseteq A: |X| \leq |N(X)|$ $\rightarrow$ I'm leaving out the proof (which is inductive)
 $\uparrow$ my subset $\uparrow$ neighbors

→ **Corollary (Frobenius):** for all k : every k -regular bip. graph contains a perfect matching. $\Rightarrow$ Graph is union of perfect matchings

→ **Theorem:** In 2^k -regular, bip. graphs, one can find a perfect matching in $O(|E|)$
 (also k -regular)

→ In bip. graphs, one can find a perfect matching in $O(|V||E|)$

Matching algorithms:

Bip. graphs: Hopcroft-Karp (unweighted): $O(|V|^{1/2} \cdot |E|)$, $O(|E|^{1+o(1)})$ for polynomial weights

Generally (with polynomial weights): Micali - Vazirani (unweighted) / Gabow - Tarjan
 $O(|V|^{1/2} \cdot |E|)$, matrix mult. - Mucha, Sankowski (unweighted): $O(|V|^{2.373})$
these guys have been trying to prove it for 30+ years

BFS for alternating paths: Input: bip. graph, Matching M , Output: (shortest) aug. path if it exists

$L_0 := \{ \text{vertices (not covered by } M) \text{ in } A \}$

Mark all vertices in L_0 as visited

for $i=1$ to n

if i odd then

$L_i := \{ \text{non-visited neighbors of } L_{i-1} \text{ through edges in } E \setminus M \}$
 else

$L_i := \{ \text{non-visited neighbors of } L_{i-1} \text{ through edges in } M \}$
 if any v in L_i not covered

then return path to v (backtracking)

Inductively: $L_i = \{ v \mid \text{shortest alternating path from } L_0 \text{ to } v \text{ has length } i \}$

Hopcroft - Karp: Input: bip. graph, Output: maximum matching M

Start with $M = \emptyset$.

repeat until $\nexists$ aug. path

- $k :=$ length of shortest aug. path

Can be found in $O(|V| + |E|)$

- find multiple disjoint aug. paths of length k until we have a maximal set S from these paths (you cannot add an aug. path of length k to S)

- for all P from S :

$M := M \oplus P$

Theorem: Hopcroft - Karp runs the repeat loop only $O(|V|^{1/2})$ times.

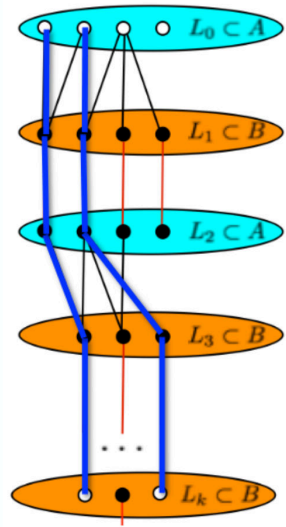
Its total runtime is $O(|V|^{1/2} \cdot (|V| + |E|))$.

A better approximation for metric TSP:

Theorem: Christofides' Algorithm computes a 1.5-approximation for the metric TSP in polynomial time (using MST, matching, and an Euler tour)

→ Karlin, Klein, Ghemawat found a $(3/2 - 10^{-36})$ -approximation

→ 1.008-approx. is NP-complete



new steps:

1. Find MST T ($L(T) \leq \text{opt}(K_n, l)$)

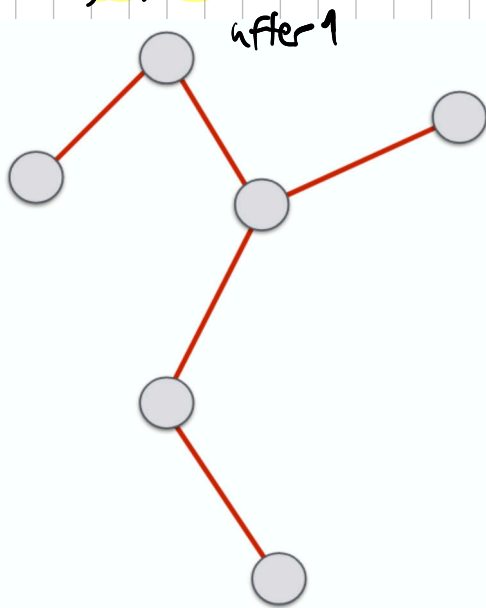
2'. $X :=$ vertices in T of odd degree, find min. matching M for X ($L(M) \leq \frac{1}{2} \text{opt}(K_n, l)$)

3. Find Euler tour W ($L(W) = L(T) + L(M) \leq \frac{3}{2} \text{opt}(K_n, l)$)

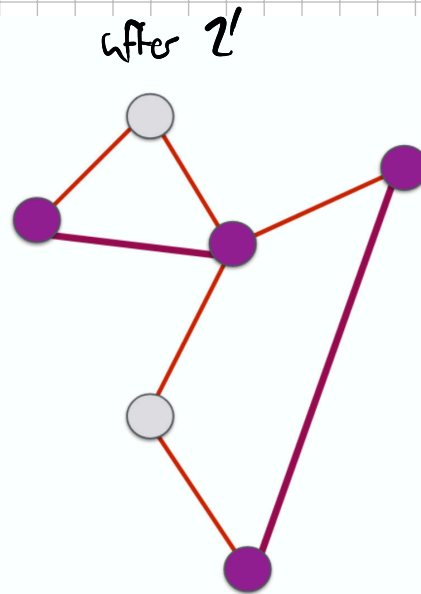
4. go through W with shortcuts, s.f. every vertex visited once $\rightarrow$ Hamilton cycle C

$$L(C) \leq L(W) = L(T) + L(M) \leq \frac{3}{2} \text{opt}(K_n, l)$$

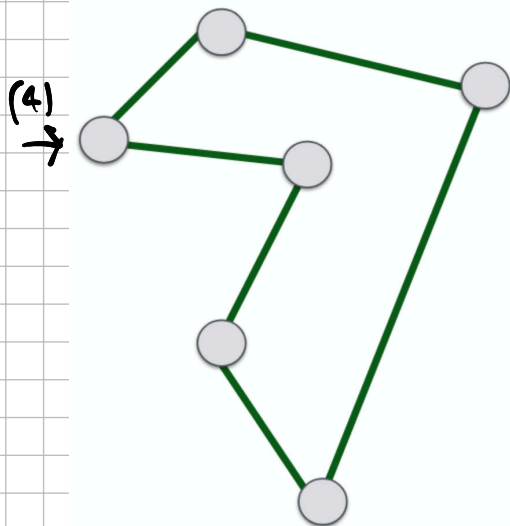
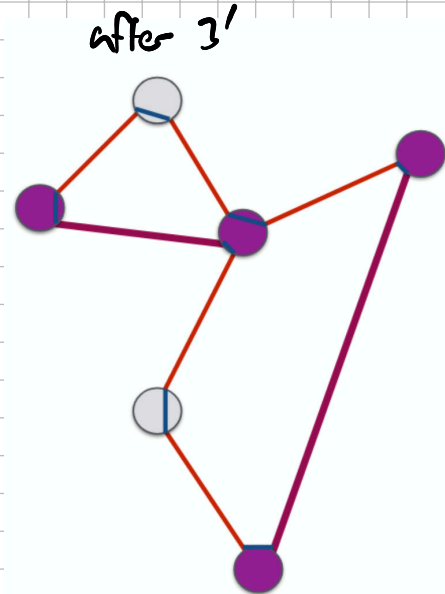
graphically:



(2')
 $\rightarrow$



(3)
 $\rightarrow$

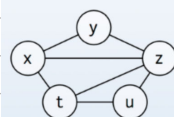


$\rightarrow$ **K-coloring**: A coloring (of a graph) with k colors is a mapping $c: V \rightarrow [k]$, so that: $c(u) \neq c(v)$ for all edges $\{u, v\} \in E$.

$\rightarrow$ cool observation that I somehow didn't get in the lecture:

this reads like an adj. matrix

Interference graph



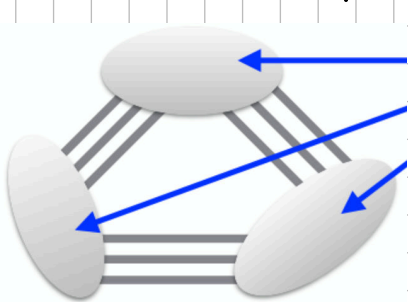
Live ranges

	x	y	z	t	u
$x \leftarrow 1$					
$y \leftarrow 2$					
$z \leftarrow x+y$					
$t \leftarrow y$					
$u \leftarrow x+t$					
print z					
print t					
print u					

omg is this pronounced like ch in Latex? please get back to me on this one

→ chromatic number $\chi(G)$ is min. amount of colors needed to color G .

→ $\chi(G) \leq k$ if and only if G k -partite



independent sets
(no edges inside them)

→ special case: $\chi(G) \leq 2$ if and only if G bipartite

→ you can find this out in $O(|E|)$ with BFS/DFS

→ for $k \geq 3$, finding out if $\chi(G) \leq k$ is NP-complete

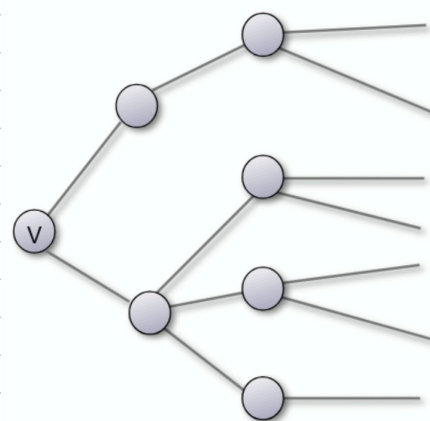
→ approximation not possible, $\forall \epsilon > 0$ $n^{1-\epsilon}$ -approx. is NP-complete

→ exponential algorithm? yes (with sieve theorem inclusion/exclusion)
→ polynomial space, time $O(2.2^n)$

→ special cases: yes

Theorem: $\forall k, r \in \mathbb{N}$: there are graphs without cycle of length $\leq k$, with $\chi(G) \geq r$

→ this practically looks like a tree



Greedy coloring (G):

1. select arbitrary order of vertices: $V = \{v_1, \dots, v_n\}$

2. $c[v_1] \leftarrow 1$

3. for $i = 2$ to $i = n$ do

4. $c[v_i] \leftarrow \min \{k \in \mathbb{N} \mid k \neq c(u) \text{ for all } u \in N(v_i) \cap \{v_1, \dots, v_{i-1}\}\}$

neighbors that came before i

→ Observation: for all orders $V = \{v_1, \dots, v_n\}$ the greedy algo.

needs at most $\Delta(G) + 1$ colors ($\Delta(G) = \max. \text{degree in } G$)

→ Observation: There is an order $V = \{v_1, \dots, v_n\}$, for which the greedy algo. only needs $\chi(G)$ colors.

→ Observation: There are bip. graphs and an order $V = \{v_1, \dots, v_n\}$, for which the greedy algo. needs $n/2$ colors.

→ Observation: If for the chosen order, $|N(v_i) \cap \{v_1, \dots, v_{i-1}\}| \leq k \quad \forall 2 \leq i \leq n$ holds, the greedy algo. maximally needs $k+1$ colors

→ Heuristic: $v_n =$ vertices of the smallest degree. delete v_n .
 $v_{n-1} =$ vertices of smallest deg. in leftover graph. delete v_{n-1} . Repeat.

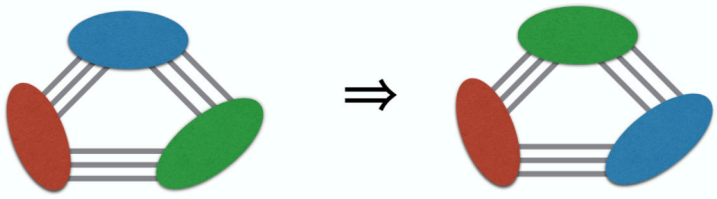
→ If in every subgraph of G , there is a vertex of degree $\leq k$, the heuristic gives us an order $\{v_1, \dots, v_n\}$ for which the greedy algo. needs at most $k+1$ colors.

→ Corollary: The heuristic always finds a 2-coloring for trees

→ Theorem: If a graph is planar (can literally be drawn without edges intersecting), there is always a vertex of degree ≤ 5 .

→ Corollary: The heuristic always finds a $\leq \Delta(G)$ -coloring for planar graphs.

→ Corollary: G is connected and $\exists v \in V$ s.t. $\deg(v) < \Delta(G)$: the heuristic or DFS/BFS finds an order s.t. the greedy algo needs at most $\Delta(G)$ colors



← we can swap classes in graphs with independent sets

← this is used in block graphs;

Presume we can color every block with k colors.

Then, we can color G with k colors.

→ every graph can be colored in $O(|E|)$ with $\Delta(G) + 1$ colors

→ Brooks' Theorem: $G \neq K_n$ (k -regular), $G = C_{2n+1}$ (complete), G connected:

$\Rightarrow G$ can be colored in $O(|E|)$ with $\Delta(G)$ colors.

Algorithm (based on Brooks' Theorem):

→ if $\Delta(G) = 2 \rightarrow$ color G with 2 colors

→ if $\exists v \in V$ with $\deg(v) < \Delta(G)$: color G with greedy algo. + heuristic

→ if $\exists$ cut vertex v : color all blocks (incl. v) with heuristic (+ color swap), s.t. v colored one certain way in all graphs

→ determine vertices $v, x, y \in V$ with $x, y \in N(v)$ and $\{x, y\} \notin E$ (as G conn., not complete)

→ Consider $G' := G[V \setminus \{x, y\}]$:

→ if G' connected: use greedy, first x and y , then heuristic for G'

→ else ...

Theorem: A 3-colorable graph can be colored in time $O(|V| + |E|)$ using $O(\sqrt{|V|})$ colors.

Algorithm for this:

While $\exists$ a vertex v with $> \sqrt{|V|}$ uncolored neighbors:

Color v with new color and its neighbors with 2 new colors

This works because the graph is tripartite, so each neighborhood $N(v)$ is bip.

Delete all colored vertices. The remaining graph has max. degree $\Delta \leq \sqrt{|V|}$.

Color the remaining vertices using the greedy algorithm with $\Delta + 1$ new colors.

Probability theory (and the start of WS-W6);

Def.: discrete probability space: determined by the set of events

$\Omega = \{\omega_1, \omega_2, \dots\}$ of elementary events ω_i . Every elementary event has a probability $\Pr[\omega_i]$ with $0 \leq \Pr[\omega_i] \leq 1$ and $\sum_{\omega \in \Omega} \Pr[\omega] = 1$. ↙ this is a "lowercase" Ω

Def.: A subset $E \subseteq \Omega$ is an event. The probability $\Pr[E]$ of an event is determined as $\Pr[E] = \sum_{\omega \in E} \Pr[\omega]$. $\bar{E} := \Omega \setminus E$ is defined as the complementary event to E .

Lemma: For events $A, B \subseteq \Omega$:

$$\rightarrow \Pr[\emptyset] = 0 \text{ and } \Pr[\Omega] = 1.$$

$$\rightarrow 0 \leq \Pr[A] \leq 1.$$

$$\rightarrow \Pr[\bar{A}] = 1 - \Pr[A]$$

$$\rightarrow \text{If } A \subseteq B, \Pr[A] \leq \Pr[B]$$

Laplace space: A finite prob. space, where all elementary events have the same probability (like picking a card, any card) ↙ probability

Observation: In a Laplace space Ω , for every event E , the following holds:

$$\Pr[E] = \frac{|E|}{|\Omega|}.$$

Warning: Combinatorics will reappear and be treated as a prerequisite in this course. Here is a non-trivial example from the lecture:

Consider the deck of cards C with $|C| = 52$. $\Omega = \{(X, Y) : X, Y \subseteq C, X \cap Y = \emptyset, |X| = |Y| = 5\}$. This is meant to emulate two players drawing 5 cards each.

Multiple Choice: Which descriptions of $|\Omega|$ are correct?

$$a) 52^5 \cdot 52^5, b) \binom{52}{10} \cdot \binom{10}{5}, c) \frac{52!}{5! \cdot 5! \cdot 42!}, d) \binom{52}{5} \cdot \binom{47}{5}$$

$\rightarrow$ a) is wrong, as 5 cards are removed from the "pool" of eligible cards after the first draw.

$\rightarrow$ b) is correct: First, you take 10 cards out of 52, then you decide how sets of 5 cards are divided between the players.

$\rightarrow$ d) is correct: remove 5 cards from the deck, then 5 from the reduced deck.

$$\rightarrow c) \text{ is correct: } \binom{52}{5} \cdot \binom{47}{5} \stackrel{\text{def. combination}}{=} \frac{52!}{5!47!} \cdot \frac{47!}{5!42!} = \frac{52!}{5!5!42!}$$

Union of events:

Addition Theorem: for pairwise disjoint events $A_1, \dots, A_n$: $\Pr\left[\bigcup_{i=1}^n A_i\right] = \sum_{i=1}^n \Pr[A_i]$

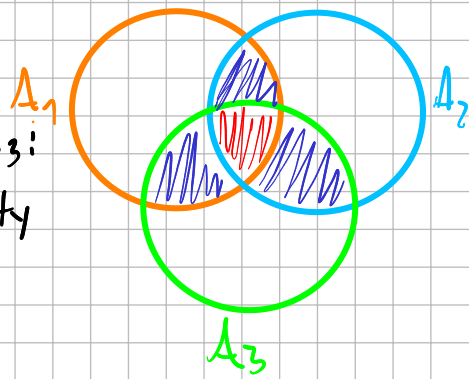
Union Bound (Theorem): For events $A_1, \dots, A_n$: $\Pr\left[\bigcup_{i=1}^n A_i\right] \leq \sum_{i=1}^n \Pr[A_i]$

Proof (optional, I found it interesting): For $i \in \{1, \dots, n\}$: $B_i = A_i \setminus (A_1 \cup A_2 \cup \dots \cup A_{i-1})$.

Then: $\Pr[B_i] \leq \Pr[A_i]$ and for $i \neq j$, B_i and B_j are disjoint.

$$Pr[\bigcup_{i=1}^n A_i] = Pr[\bigcup_{i=1}^n B_i] = \sum_{i=1}^n Pr[B_i] \stackrel{\text{subset}}{\leq} \sum_{i=1}^n Pr[A_i]$$

Addition Theorem



Inclusion/Exclusion Theorem: Intuition: Consider events A_1, A_2, A_3 :
 → How do we avoid double counting and just get the probability of their disjoint union?

$$Pr[A_1 \cup A_2 \cup A_3] = Pr[A_1] + Pr[A_2] + Pr[A_3] - Pr[A_1 \cap A_2] - Pr[A_1 \cap A_3] - Pr[A_2 \cap A_3] + Pr[A_1 \cap A_2 \cap A_3]$$

Theorem: For events $A_1, \dots, A_n$ ($n \geq 2$): $Pr[\bigcup_{i=1}^n A_i] = \sum_{k=1}^n (-1)^{k+1} \cdot \sum_{1 \leq i_1 < \dots < i_k \leq n} Pr[A_{i_1} \cap \dots \cap A_{i_k}]$
 $= \sum_{i=1}^n Pr[A_i] - \sum_{1 \leq i_1 < i_2 \leq n} Pr[A_{i_1} \cap A_{i_2}] + \dots - \dots + \dots + (-1)^{n+1} \cdot Pr[A_1 \cap \dots \cap A_n]$

Example: Consider a deck of n cards: $A_i =$ "There is ~~a light and it never goes out~~ a card that is in the same position after mixing the deck." Determine $Pr[A]$.

Idea: $A_i =$ "Card i in the same position as before mixing" $\Rightarrow A = \bigcup_{i=1}^n A_i \Rightarrow$ inclusion/exclusion
 For $S \subseteq [n]$: $A_S =$ "A holds for all cards in S " $= \bigcap_{i \in S} A_i \Rightarrow Pr[A_S] = \frac{(n-|S|)!}{n!}$

Explaining $\frac{(n-|S|)!}{n!}$: we fix $|S|$ cards in their positions, only $n-|S|$ cards can move
 ← number of outcomes, all permutations are equally likely

Using inclusion/exclusion:

$$Pr[\bigcup_{i=1}^n A_i] = \sum_{i=1}^n Pr[A_i] - \sum_{S \subseteq [n], |S|=2} Pr[A_S] + \sum_{S \subseteq [n], |S|=3} Pr[A_S] - \dots$$

$$= n \cdot \frac{(n-1)!}{n!} - \binom{n}{2} \cdot \frac{(n-2)!}{n!} + \binom{n}{3} \cdot \frac{(n-3)!}{n!} - \dots$$

$$= 1 - \frac{1}{2!} + \frac{1}{3!} - \dots \approx \frac{1}{n!} \approx 1 - \frac{1}{e} \approx 63.2\%$$

$$\frac{\binom{n}{k} \cdot \frac{(n-k)!}{n!}}{\frac{n!}{k! (n-k)!} \cdot \frac{(n-k)!}{n!}} = \frac{1}{k!}$$

Conditional probabilities:

Def.: Let A and B be events with $Pr[B] > 0$. The ^{conditional} cond. prob. $Pr[A|B]$ of A given B is defined as $Pr[A|B] := \frac{Pr[A \cap B]}{Pr[B]}$.

The following hold: $Pr[A|\Omega] = Pr[A]$, $Pr[A|A] = 1$, $Pr[A|\bar{A}] = 0$,
 $Pr[A \cap B] = Pr[A|B] \cdot Pr[B] = Pr[B|A] \cdot Pr[A]$

Multiplication theorem: Let $A_1, \dots, A_n$ be events. If $Pr[A_1 \cap \dots \cap A_n] > 0$,
 $Pr[A_1 \cap \dots \cap A_n] = Pr[A_1] \cdot Pr[A_2|A_1] \cdot Pr[A_3|A_1 \cap A_2] \dots Pr[A_n|A_1 \cap \dots \cap A_{n-1}]$

Proof (also optional): $P_r[A_1] \geq P_r[A_1 \cap A_2] \geq \dots \geq P_r[A_1 \cap \dots \cap A_n] > 0$

$\Rightarrow$ all probs. well-defined $\Rightarrow$ apply def. cond. prob.

$$\cancel{P_r[A_1]} \cdot \frac{P_r[A_1 \cap A_2]}{\cancel{P_r[A_1]}} \cdot \frac{P_r[A_1 \cap A_2 \cap A_3]}{\cancel{P_r[A_1 \cap A_2]}} \cdot \dots \cdot \frac{P_r[A_1 \cap \dots \cap A_n]}{\cancel{P_r[A_1 \cap \dots \cap A_{n-1}]}} \quad (\text{telescoping product!})$$

Non-trivial application: Birthday problem:

Consider a room of n people. Determine the probability of 2 people sharing a birthday.

Reshape this: we have n balls and n bins. Here, $n = 365$. Determine the prob. of 2 balls landing in the same bin  / the prob. of all balls landing in different bins.

Event A_i : " i -th ball lands in a "free" bin". $\rightarrow P_r[A_1] = 1, P_r[A_2|A_1] = \frac{n-1}{n} = 1 - \frac{1}{n},$
 $P_r[A_i|A_1 \cap \dots \cap A_{i-1}] = \frac{n-(i-1)}{n} = 1 - \frac{i-1}{n}$

Mult. theorem: $P_r[A_1 \cap \dots \cap A_n] = 1 \cdot (1 - \frac{1}{n}) \cdot (1 - \frac{2}{n}) \cdot \dots \cdot (1 - \frac{n-1}{n})$ ^{a constant}

Estimating the products: $1-x \approx e^{-x}$ if x small (more accurately: $1-x = e^{-x(1-O(x))}$
 $1-x \leq e^{-x} \forall x \in \mathbb{R}$)

$$(1 - \frac{1}{n})(1 - \frac{2}{n}) \dots (1 - \frac{n-1}{n}) \approx e^{-\frac{1}{n}} \cdot e^{-\frac{2}{n}} \cdot \dots \cdot e^{-\frac{n-1}{n}} \approx e^{-\frac{1}{n}(1+2+\dots+(n-1))} \approx e^{-\frac{(n-1) \cdot n}{2n}} = e^{-\frac{n-1}{2}}$$

Theorem of total probability: Let $A_1, \dots, A_n$ be pairwise disjoint events and let $B \subseteq A_1 \cup \dots \cup A_n$. Then, $P_r[B] = \sum_{i=1}^n P_r[B|A_i] \cdot P_r[A_i]$.

Application (goat problem 

There are 3 doors: there is a goat behind 2 of them and a car behind 1. The candidate chooses a door and the host opens a door that has a goat behind it (not the one chosen). The candidate is offered the chance to change doors, should she do it?

Using the theorem: A_1 = "Candidate chose the correct door in the beginning",

A_2 = "She chose the wrong door", B = "She wins if she switches doors"

Keep in mind that by the theorem, $P_r[B] = \sum_{i=1}^n P_r[B|A_i] \cdot P_r[A_i]$.

$$P_r[B|A_1] = 0, P_r[B|A_2] = 1, P_r[A_1] = \frac{1}{3}, P_r[A_2] = \frac{2}{3}. \Rightarrow P_r[B] = 0 \cdot \frac{1}{3} + 1 \cdot \frac{2}{3} = \frac{2}{3}.$$

Bayes' Theorem: Let $A_1, \dots, A_n$ be disjoint events and $B \subseteq A_1 \cup \dots \cup A_n$ be an event with

$$P_r[B] > 0. \text{ Then, } \forall i \in \{1, \dots, n\} \quad P_r[A_i|B] = \frac{P_r[B|A_i] \cdot P_r[A_i]}{\sum_{j=1}^n P_r[B|A_j] \cdot P_r[A_j]}$$

Proof: $P_r[B|A_i] \cdot P_r[A_i] = P_r[B \cap A_i], \sum_{i=1}^n P_r[B|A_i] \cdot P_r[A_i] \overset{\text{Th. of total prob.}}{=} P_r[B] \Rightarrow$ true by def. of cond. prob.

Example: Test for a disease (we had this exact one at school)

$A_1 =$ "you have the disease", $A_2 =$ "you don't have it", $B =$ "Test positive"

Known: Sensitivity $\rightarrow P[B|A_1] = 72\%$, Specificity $\rightarrow P[B|A_2] = 98\%$

$$P[A_1|B] = \frac{P[B|A_1] \cdot P[A_1]}{P[B|A_1] \cdot P[A_1] + P[B|A_2] \cdot P[A_2]} = \frac{0.72 \cdot P[A_1]}{0.72 \cdot P[A_1] + 0.02 \cdot P[A_2]} \approx \begin{cases} 26.7\% & \text{if } P[A_1] = 1\% \\ 0.72\% & \text{if } P[A_1] = 0.02\% \end{cases}$$

Independence:

intuitively: $P[A|B]$ gives $P[A]$ under the condition of B . A and B are independent, if B coming into effect does not change $P[A]$, so $\frac{P[A \cap B]}{P[B]} = P[A|B] = P[A]$

Def.: Events A and B are independent if $P[A \cap B] = P[A] \cdot P[B]$

Example: Roll 2 dice. $A :=$ "Sum of face values is 7", $B :=$ "The second die's face value is 2"

$$P[A] = \frac{6}{36}, P[B] = \frac{1}{6}, P[A \cap B] = \frac{3}{36} = P[A] \cdot P[B] \Rightarrow \text{independent}$$

Independence of multiple events:

Def.: Events $A_1, \dots, A_n$ are independent if for all subsets $J \subseteq \{1, \dots, n\}$, $P[\bigcap_{j \in J} A_j] = \prod_{j \in J} P[A_j]$ (1)

An infinite group of events A_i with $i \in \mathbb{N}$ is independent if (1) holds for all finite subsets $J \subseteq \mathbb{N}$.

Lemma: Let $A_1, \dots, A_n$ be events, A_i^0 be $\bar{A}_i$ and A_i^1 be A_i ($\forall i \in \{1, \dots, n\}$). Then, $A_1, \dots, A_n$ are independent if and only if for all $(s_1, \dots, s_n) \in \{0, 1\}^n$: $P[\bigcap_{j=1}^n A_j^{s_j}] = \prod_{j=1}^n P[A_j^{s_j}]$

$\rightarrow$ The proof is omitted to save space and time, but it is available in the **Lecture 11** notes on Moodle and worth understanding

Misconception: $A_1, \dots, A_n$ are independent if $P[A_1, \dots, A_n] = \prod_{i=1}^n P[A_i]$ $\leftarrow$ this is false!

Lemma: Let A, B, C be independent. Then, 1) $A \cap B$ and C are independent, 2) $A \cup B$ and C are independent.

Proof of the lemma: 1) $P[(A \cap B) \cap C] = P[A] \cdot P[B] \cdot P[C] = P[A \cap B] \cdot P[C]$

$$\begin{aligned} 2) P[(A \cup B) \cap C] &= P[(A \cap C) \cup (B \cap C)] \stackrel{\text{Sieve formula}}{=} P[A \cap C] + P[B \cap C] - P[A \cap B \cap C] \\ &\stackrel{\text{ind. of } A, B, C}{=} P[C] \cdot (P[A] + P[B] - P[A \cap B]) \end{aligned}$$

Random variables:

Def.: A random variable is a map $X: \Omega \rightarrow \mathbb{R}$, with Ω being the outcome range of a probability space.

Example: Dice $\rightarrow \Omega = \{1, 2, 3, 4, 5, 6\}$

$$X(\omega) = \begin{cases} 1 & \text{if } \omega \text{ prime} \\ 0 & \text{else} \end{cases} \quad (\text{Description as Bernoulli trial}), \text{ or } X(\omega) = \omega^2 - 5\omega \quad (\text{Polynomial description})$$

Note: The polynomial does not have to be meaningful, it just needs to be a function

Def.: the range of a random variable X is $W_X := X(\Omega) = \{a \in \mathbb{R} \mid \exists \omega \in \Omega \text{ with } X(\omega) = a\}$

$\rightarrow$ Range is defined the same as for functions in Calculus $\rightarrow$ set of all possible values of X

Notation: " $X \leq 5$ " describes the event $\{\omega \in \Omega : X(\omega) \leq 5\} \rightarrow$ the event that X "returns" a value ≤ 5 .

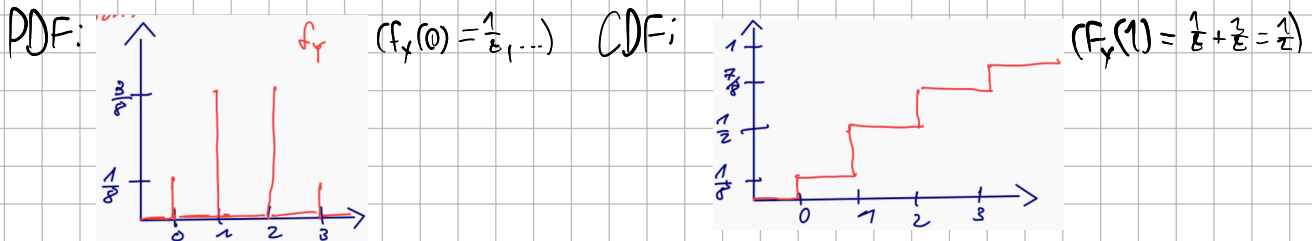
Probability Density Function (PDF): $f_X: \mathbb{R} \rightarrow [0, 1]$
 $\alpha \rightarrow \Pr[X = \alpha]$ [I got carried away, W7-W8 starts here]

Cumulative Distribution Function (CDF): $F_X: \mathbb{R} \rightarrow [0, 1]$
 $\alpha \rightarrow \Pr[X \leq \alpha]$

Example: Roll a die thrice. Probability Space $\Omega = \{1, \dots, 6\}^3$

Random var. $Y =$ amount of odd face values rolled

$$\Pr[Y=0] = \frac{1}{8}, \Pr[Y=1] = \frac{3}{8}, \Pr[Y=2] = \frac{3}{8}, \Pr[Y=3] = \frac{1}{8}$$



Expected value:

Def.: For a random variable X with range W_X , the expected value $\mathbb{E}[X]$ is defined as $\mathbb{E}[X] := \sum_{\alpha \in W_X} \alpha \cdot \Pr[X = \alpha]$ if the sum converges absolutely. Else, the expected value is undefined. (This case will not be applied in this course)

Lemma: for a random variable X , $\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega) \cdot \Pr[\omega]$

$\rightarrow$ this is near equivalent to the def.

Special case $W_X \subseteq \mathbb{N}_0$:

Theorem: Let X be a random variable with $W_X \subseteq \mathbb{N}_0$. Then: $\mathbb{E}[X] = \sum_{i=1}^{\infty} \Pr[X \geq i]$

Proof: $\mathbb{E}[X] = \sum_{i=0}^{\infty} i \cdot \Pr[X = i] = \sum_{i=0}^{\infty} \sum_{j=1}^i \Pr[X = i] = \sum_{j=1}^{\infty} \sum_{i=j}^{\infty} \Pr[X = i] = \sum_{j=1}^{\infty} \Pr[X \geq j]$
sum over pairs (i, j) with $i \geq j \geq 1$

Example (typical): Toss a coin with $\Pr[\text{"Heads"}] = p$ ($0 < p < 1$), $X =$ amount of throws until first "Heads". $\mathbb{E}[X] = \sum_{i=1}^{\infty} \Pr[X \geq i] = \sum_{i=1}^{\infty} (1-p)^{i-1} = \sum_{i=0}^{\infty} (1-p)^i = \frac{1}{p}$
 $\Pr[\text{"first } i-1 \text{ tosses are Tails"}]$

Indicator variables: For an event $A \subseteq \Omega$, the indicator variable (ind. var.) X_A is defined as

$$X_A(\omega) := \begin{cases} 1 & \text{if } \omega \in A \\ 0 & \text{else} \end{cases}$$

Observation: for the ind. var. X_A of an event A : $\mathbb{E}[X_A] = \Pr[A]$

Linearity of the expected value:

Theorem: For random variables $X_1, \dots, X_n$ and $X := a_1 \cdot X_1 + \dots + a_n \cdot X_n + b$

with $a_1, \dots, a_n, b \in \mathbb{R}$: $\mathbb{E}[X] = a_1 \cdot \mathbb{E}[X_1] + \dots + a_n \cdot \mathbb{E}[X_n] + b$

→ "E of the sum = sum of expected values"

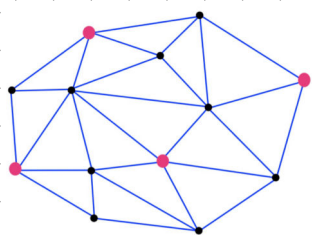
Proof: $E[X] = \sum_{\omega \in \Omega} X(\omega) \cdot P[\omega] = \sum_{\omega \in \Omega} (a_1 \cdot X_1(\omega) + \dots + a_n \cdot X_n(\omega) + b) \cdot P[\omega]$
 $= a_1 \cdot \underbrace{\left(\sum_{\omega \in \Omega} X_1(\omega) \cdot P[\omega] \right)}_{E[X_1]} + \dots + a_n \underbrace{\left(\sum_{\omega \in \Omega} X_n(\omega) \cdot P[\omega] \right)}_{E[X_n]} + b \cdot \underbrace{\left(\sum_{\omega \in \Omega} P[\omega] \right)}_1$

Example: We toss a coin thrice. X_i = amount of partial sequences that consist of 3 times heads, $E[X]$?

$X = X_1 + \dots + X_{m-2}$ with $X_i = \begin{cases} 1 & \text{if at position } i, \text{ the partial sequence HHH begins} \\ 0 & \text{else} \end{cases}$

$E[X] = P[X_i = 1] = \frac{1}{8} \Rightarrow E[X] = \sum_{i=1}^{m-2} E[X_i] = \frac{m-2}{8}$
 ↑ Linearity of E

Practical Use: Stable Set



Networking: each vertex is a computer, we want to find the maximum stable set

→ stable set: vertices not connected by edges

Goal: Every computer v computes $S_v \in \{0, 1\}$ s.t. $S = \{v \in V : S_v = 1\}$ is

a stable set.

Algorithm: ① Set $S_v \leftarrow 1$ with probability p and $S_v \leftarrow 0$ else

② For all $u \in V$ with $\{u, v\} \in E$:

If: $S_u = S_v = 1$: Set one of S_u and S_v to 0
 ↑ guarantees correctness

} Information exchange between neighbors

③ Return: S_v

Theorem: The algorithm computes a stable set S with $E[|S|] \geq n \cdot p - m \cdot p^2$ ($n = |V|$, $m = |E|$)

X := amount of vertices v with $S_v = 1$ following ①, Y := amount of edges considered in ②

Observation: $|S| \geq X - Y$

$e = \{u, v\}$ is considered if $S_u = S_v = 1$ after ①

Linearity of E: $\Rightarrow E[|S|] \geq E[X] - E[Y]$

Determine $E[X]$: X_v := ind. var. for " $S_v = 1$ after ①"

Then: $X = \sum_{v \in V} X_v \Rightarrow E[X] = \sum_{v \in V} E[X_v] = n \cdot p$
 (= p)

$E[Y]$: Y_e := ind. var. for "edge e considered in ②"

Then: $Y = \sum_{e \in E} Y_e \Rightarrow E[Y] = \sum_{e \in E} E[Y_e] = m \cdot p^2$

$E[Y_e] = P[e \text{ considered in ②}]$
 $= P[S_u = 1 \text{ and } S_v = 1 \text{ after ①}]$
 $= P[S_u = 1 \text{ after ①}] \cdot P[S_v = 1 \text{ after ①}]$
 $= p \cdot p$

Optimal choice of p : $p = \frac{2}{m}$, then $E[|S|] \geq \frac{n^2}{4m} \geq \frac{n}{2k}$ ($m = \frac{k \cdot n}{2}$)
 ↑
 for k -regular graphs

Variance:

Def.: For a random variable X with $\mu = E[X]$, the variance $\text{Var}[X]$ is defined as:

$$\text{Var}[X] := E[(X - \mu)^2] \text{ with } \sigma = \sqrt{\text{Var}[X]} \text{ is the standard deviation of } X.$$

Example: $P[X = 10^{10}] = \frac{1}{2} = P[X = -10^{10}] \Rightarrow \mu = 0, \text{Var}[X] = 10^{20}, \sigma = 10^{10}$

Theorem: For any random variable X , $\text{Var}[X] = E[X^2] - E[X]^2$.

Proof: $\text{Var}[X] = E[(X - \mu)^2] = E[X^2 - 2\mu X + \mu^2] \stackrel{\text{linearity of } E}{=} E[X^2] - 2\mu E[X] + \mu^2 \stackrel{\mu = E[X]}{=} E[X^2] - (E[X])^2$

Theorem: For any random variable X and $a, b \in \mathbb{R}$: $\text{Var}[a \cdot X + b] = a^2 \cdot \text{Var}[X]$.

Proof: ① $\text{Var}[X+b] = \text{Var}[X]$: Lin. of E : $E[X+b] = E[X] + b \Rightarrow \text{Var}[X+b] = E[(X+b - E[X+b])^2] = E[(X - E[X])^2] = \text{Var}[X]$

② $\text{Var}[a \cdot X] = a^2 \cdot \text{Var}[X]$: $\text{Var}[a \cdot X] = E[(a \cdot X)^2] - E[a \cdot X]^2 = a^2 E[X^2] - (a \cdot E[X])^2 \stackrel{\text{Theorem}}{=} a^2 \cdot \text{Var}[X]$

Moments:

Def.: For a random variable X , $E[X^k]$ is the k -th moment and $E[(X - E[X])^k]$ is the k -th central moment.

First moment: $E[X]$, **first central moment** $= E[X] - E[X] = 0$, **2nd central moment:** $E[(X - E[X])^2] = E[X^2] - (E[X])^2 = \text{Var}[X]$

Important Distributions:

Def.: A random variable X with range $W_X = \{0, 1\}$ and PDF $f_X(x) = \begin{cases} p & \text{for } x=1 \\ 1-p & \text{for } x=0 \\ 0 & \text{else} \end{cases}$

is **Bernoulli distributed** with probability of success $= p$, which is written as $X \sim \text{Bernoulli}(p)$

$\rightarrow$ This describes binary events, such as various ind. vars.

$\rightarrow E[X] = p, \text{Var}[X] = E[X^2] - E[X]^2 = p - p^2 = p(1-p)$

Def.: A random variable X with range $W_X = \{0, \dots, n\}$ and PDF $f_X(x) = \begin{cases} \binom{n}{k} \cdot p^k \cdot (1-p)^{n-k} & \text{for } k \in \{0, \dots, n\} \\ 0 & \text{else} \end{cases}$ is **binomially distributed** with parameters p and n , written $X \sim \text{Bin}(n, p)$

Example: Toss a coin n times, with $P[\text{"Heads"}] = p, P[\text{"Tails"}] = 1-p$, $X :=$ amount of heads thrown

$P[X=k] = \binom{n}{k} p^k (1-p)^{n-k} \Rightarrow n$ positions, depending on results of n tosses $\Rightarrow$ choose k positions that are heads $\Rightarrow \binom{n}{k}$ possibilities

Prob. that n throws have exactly this result: $p^k (1-p)^{n-k}$

Def.: A random variable with PDF $f_X(i) = \begin{cases} \frac{e^{-\lambda} \lambda^i}{i!} & \text{for } i \in \mathbb{N}_0 \\ 0 & \text{else} \end{cases}$ is **Poisson distributed** with parameter λ , written as $X \sim \text{Po}(\lambda)$

$\rightarrow \text{Bin}(n, \frac{\lambda}{n})$ converges to $\text{Po}(\lambda)$ for $n \rightarrow \infty, E[X] = \text{Var}[X] = \lambda$

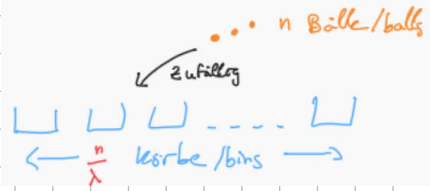
Proof of these observations:

X_i := amount of balls in first bin $\sim \text{Bin}(n, \frac{\lambda}{n})$

$$\Rightarrow E[X] = n \cdot \frac{\lambda}{n} = \lambda \quad \text{w/ } (e^{-\frac{\lambda}{n}})^{n-i} = e^{-\lambda \frac{n-i}{n}} = e^{-\lambda}$$

$$\lim_{n \rightarrow \infty} P[X=i] = \lim_{n \rightarrow \infty} \binom{n}{i} \left(\frac{\lambda}{n}\right)^i \left(1 - \frac{\lambda}{n}\right)^{n-i} = \frac{\lambda^i}{i!} e^{-\lambda}$$

$$= \frac{\lambda^i}{i!} \left(\frac{n}{n} \cdot \frac{(n-1)}{n} \cdot \dots \cdot \frac{(n-i+1)}{n}\right)$$



Use case: Describing rare events, such as X = "amount of heart attacks in in the next hour"

Def.: A random variable X with PDF $f_X(i) = \begin{cases} p \cdot (1-p)^{i-1} & \text{for } i \in \mathbb{N} \\ 0 & \text{else} \end{cases}$ is geometrically distributed with p = probability of success, written as $X \sim \text{Geo}(p)$

$$\rightarrow E[X] = \frac{1}{p}, \text{Var}[X] = \frac{1-p}{p^2}, \text{CDF: } F_X(n) = P[X \leq n] = \sum_{i=1}^n P[X=i] = \sum_{i=1}^n p \cdot (1-p)^{i-1} = 1 - (1-p)^n$$

Example: Toss a coin n times with $P[\text{"Heads"}] = p$, $P[\text{"Tails"}] = 1-p$ until we have gotten heads once,

X := Amount of tosses.

$$P[X=k] = \underbrace{(1-p)^{k-1}}_{\text{Tails } k-1 \text{ times, then heads}} \cdot p$$

Theorem: If $X \sim \text{Geo}(p)$, $\forall s, t \in \mathbb{N}$: $P[X \geq s+t | X \geq s] = P[X \geq t]$

Def.: for $n \geq 2$, a random variable X with density $f_X(k) = \begin{cases} \binom{k-1}{n-1} \cdot p^n \cdot (1-p)^{k-n} & \text{for } k=1, 2, \dots \\ 0 & \text{else} \end{cases}$

is negatively binomial distributed with order n

$$\rightarrow E[X] = \frac{n}{p}$$

X_i := amount of coin tosses after $(i-1)$ th success up to and including i -th success

$$\text{Then: } X_i \sim \text{Geo}(p), X = \sum_{i=1}^n X_i \Rightarrow E[X] = \sum_{i=1}^n E[X_i] = n \cdot \frac{1}{p} = \frac{n}{p}$$

Example (waiting for n -th success): Toss a coin n times with $P[\text{"Heads"}] = p$, $P[\text{"Tails"}] = 1-p$ until we get heads

n times, X := amount of tosses

$$P[X=k] = \binom{k-1}{n-1} (1-p)^{k-n} \cdot p^n$$

$\rightarrow k$ th toss = heads, select $n-1$ other tosses that would have shown heads $\rightarrow \binom{k-1}{n-1}$ possibilities

$\rightarrow P[\text{"the } k \text{ tosses have same result"}] : (1-p)^{k-n} \cdot p^n$

Multiple Random Variables:

Let X, Y be random variables:

joint PDF: $f_{X,Y}(\alpha, \beta) = P[X=\alpha, Y=\beta]$, edge density of X (= PDF of X): $f_X(\alpha) = \sum_{\beta \in \Omega_Y} f_{X,Y}(\alpha, \beta)$

$$= \sum_{\beta \in \Omega_Y} P[X=\alpha, Y=\beta] = P[X=\alpha]$$

Example: Cast a die. X := face value of die, after this, toss a coin X times. Y := amount of heads.

$$f_{X,Y}(n, k) = P[X=n, Y=k] = \begin{cases} \frac{1}{6} \binom{n}{k} \cdot \frac{1}{2^n} & \text{if } n \in \{1, \dots, 6\}, k \in \{0, 1, \dots, n\} \\ 0 & \text{else} \end{cases}$$

$$P[Y=k] = f_Y(k) = \sum_{n=\max\{1, k\}}^6 \sum_{\text{else}} \frac{1}{6} \binom{n}{k} \frac{1}{2^n} \quad \text{if } k \in \{0, 1, \dots, 6\}$$

Independence of random variables:

Def.: Random variables $X_1, \dots, X_n$ independent if and only if $\forall (\alpha_1, \dots, \alpha_n) \in W_{X_1} \times \dots \times W_{X_n}$:

$$\underbrace{\Pr[X_1 = \alpha_1, \dots, X_n = \alpha_n]}_{f_{X_1, \dots, X_n}(\alpha_1, \dots, \alpha_n)} = \underbrace{\Pr[X_1 = \alpha_1]}_{f_{X_1}(\alpha_1)} \cdot \dots \cdot \underbrace{\Pr[X_n = \alpha_n]}_{f_{X_n}(\alpha_n)}$$

→ "joint PDF = product of edge densities"

Observation: Events $A_1, \dots, A_n$ are independent if and only if indicator variables $I_{A_1}, \dots, I_{A_n}$ are independent.

Lemma: Let $X_1, \dots, X_n$ be independent random variables and $S_1, \dots, S_n \subseteq \mathbb{R}$ any sets:

$$\Pr[X_1 \in S_1, \dots, X_n \in S_n] = \Pr[X_1 \in S_1] \cdot \dots \cdot \Pr[X_n \in S_n]$$

Proof: Assume $S_i \subseteq W_{X_i} \forall i \in [n]$, S_i countable:

$$\Pr[X_1 \in S_1, \dots, X_n \in S_n] = \sum_{\alpha_1 \in S_1} \dots \sum_{\alpha_n \in S_n} \Pr[X_1 = \alpha_1, \dots, X_n = \alpha_n]$$

$$\begin{aligned} \text{Ind. of } X_1, \dots, X_n &\Rightarrow \sum_{\alpha_1 \in S_1} \dots \sum_{\alpha_n \in S_n} \Pr[X_1 = \alpha_1] \cdot \dots \cdot \Pr[X_n = \alpha_n] \\ &= \underbrace{\left(\sum_{\alpha_1 \in S_1} \Pr[X_1 = \alpha_1] \right)}_{\Pr[X_1 \in S_1]} \cdot \dots \cdot \underbrace{\left(\sum_{\alpha_n \in S_n} \Pr[X_n = \alpha_n] \right)}_{\Pr[X_n \in S_n]} \end{aligned}$$

Corollary: Let $X_1, \dots, X_n$ be independent random variables and $I \subseteq [n]$. Then, the random variables $(X_i)_{i \in I}$ are also independent.

Proof: Let $I = \{i_1, \dots, i_k\}$ and $(\alpha_{i_1}, \dots, \alpha_{i_k}) \in W_{X_{i_1}} \times \dots \times W_{X_{i_k}}$.

For $i \in [n]$, let $S_i = \begin{cases} \{\alpha_i\} & \text{if } i \in I \\ W_{X_i} & \text{else} \end{cases}$

$$\begin{aligned} \text{Then, } \Pr[X_{i_1} = \alpha_{i_1}, \dots, X_{i_k} = \alpha_{i_k}] &\stackrel{\text{Lemma}}{=} \Pr[X_1 \in S_1, \dots, X_n \in S_n] \\ &\stackrel{\text{Lemma}}{=} \Pr[X_1 \in S_1] \cdot \dots \cdot \Pr[X_n \in S_n] \\ &= \Pr[X_{i_1} = \alpha_{i_1}] \cdot \dots \cdot \Pr[X_{i_k} = \alpha_{i_k}] \end{aligned}$$

Theorem: Let $f_1, \dots, f_n$ be functions with $f_i: \mathbb{R} \rightarrow \mathbb{R}$ for $i = 1, \dots, n$. If $X_1, \dots, X_n$ are independent, this holds for $f_1(X_1), \dots, f_n(X_n)$.

Proof: For $i = 1, \dots, n$ consider values $\alpha_1, \dots, \alpha_n$ with $\alpha_i \in W_{f_i(X_i)}$

$$S_i = \{\beta: f_i(\beta) = \alpha_i\}$$

$$\begin{aligned} \text{Then: } \Pr[f_1(X_1) = \alpha_1, \dots, f_n(X_n) = \alpha_n] &\stackrel{\text{Lemma}}{=} \Pr[X_1 \in S_1, \dots, X_n \in S_n] \\ &\stackrel{\text{independence}}{\rightarrow} \Pr[X_1 \in S_1] \cdot \dots \cdot \Pr[X_n \in S_n] \\ &\rightarrow \Pr[f_1(X_1) = \alpha_1] \cdot \dots \cdot \Pr[f_n(X_n) = \alpha_n] \end{aligned}$$

Combined Random Variables:

Theorem: For two independent random variables X and Y , let $Z := X + Y$:

$$f_z(\alpha) = \sum_{\beta \in W_X} f_X(\beta) \cdot f_Y(\alpha - \beta).$$

Proof: $f_z(\alpha) = P[X+Y=\alpha] = \sum_{\beta \in W_X} P[X=\beta \text{ and } Y=\alpha-\beta]$

independence $\rightarrow = \sum_{\beta \in W_X} \underbrace{P[X=\beta]}_{f_X(\beta)} \cdot \underbrace{P[Y=\alpha-\beta]}_{f_Y(\alpha-\beta)}$

Corollary: If $X \sim P_0(\lambda)$, $Y \sim P_0(\mu)$, X, Y are independent, then $X+Y \sim P_0(\lambda+\mu)$

Moments of Combined Random Variables:

Theorem: For independent random variables $X_1, \dots, X_n$: $E[X_1 \cdot \dots \cdot X_n] = E[X_1] \cdot \dots \cdot E[X_n]$.

Theorem: For independent random variables $X_1, \dots, X_n$: $\text{Var}[X_1 + \dots + X_n] = \text{Var}[X_1] + \dots + \text{Var}[X_n]$.

$\rightarrow$ I have omitted the proofs from the lecture because they do not prove the general case.

Wald's Identity:

Theorem: Let N and X be two independent random variables with $W_N \subseteq \mathbb{N}$.

Furthermore, let $Z := \sum_{i=1}^N X_i$, with $X_1, X_2, \dots$, being independent copies of X .

Then, $E[Z] = E[N] \cdot E[X]$.

Example: Cast a die, let $N :=$ face value of die. Then, throw a coin N times.

Let $Z :=$ amount of heads. What is $E[Z]$? $E[Z] = \underbrace{3.5}_{E[N]} \cdot \underbrace{\frac{1}{2}}_{E[\text{heads on a coin toss}]}$

Conditional Random Variables:

X = random variable, A = event with $P[A] > 0$.

$X|A \Rightarrow$ Prob. of X taking on a value conditional on A .

$$\Rightarrow P[X|A = \alpha] := P[X = \alpha | A], E[X|A] = \sum_{\alpha \in W_X} \alpha \cdot P[X = \alpha | A]$$

Markov Inequality:

Theorem: Let X be a random var. that does not take on negative values. Then, for all $t \in \mathbb{R}$ with $t > 0$, $P[X \geq t] \leq \frac{E[X]}{t}$, or equivalently: $P[X \geq t \cdot E[X]] \leq \frac{1}{t}$

Diagram:



$\rightarrow$ this is used to upper bound probabilities given $E[X]$.

Proof: take X = non-neg. random var. / $t > 0$: (to show: $P[X \geq t] \leq \frac{E[X]}{t}$)

$$E[X] = \sum_{\alpha \in W_X} \alpha \cdot P[X = \alpha]$$

$$= \underbrace{\sum_{\alpha < t} \alpha \cdot P[X = \alpha]}_{> 0} + \sum_{\alpha \geq t} \alpha \cdot P[X = \alpha]$$

$$\geq t \cdot \sum_{\substack{a \in \mathcal{X} \\ a \geq t}} \Pr[X=a] = t \cdot \Pr[X \geq t]$$

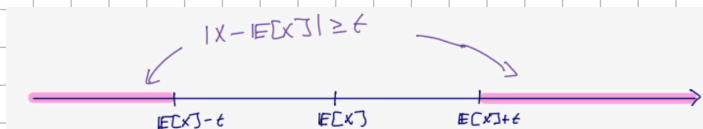
Chebyshev Inequality:

Theorem: Let X be a random variable and $t \in \mathbb{R}$ with $t > 0$.

Then, $\Pr[|X - \mathbb{E}[X]| \geq t] \leq \frac{\text{Var}[X]}{t^2}$ or equivalently, $\Pr[|X - \mathbb{E}[X]| \geq t \cdot \sigma] \leq \frac{1}{t^2}$ ($\sigma := \sqrt{\text{Var}[X]}$)

Proof: left out of the sheet, it uses Markov on $Y := (X - \mathbb{E}[X])^2$

Diagram:



EXAMPLE from FS17: Let $n \in \mathbb{N}$, and let X be a random var. with $\mathbb{E}[X] = n \log n$ and $\text{Var}[X] = n^2$. Show: $\exists C > 0$ s.t. $\Pr[X \geq n \log n + C \cdot n] \leq 0.01$.

$$\begin{aligned} \text{By Chebyshev, } \Pr[T_n \geq n \log n + C \cdot n] &\leq \Pr[|T_n - n \log n| \geq C \cdot n] \\ &= \Pr[|T_n - \mathbb{E}[T_n]| \geq C \cdot n] \\ &\leq \frac{\text{Var}[T_n]}{(C \cdot n)^2} = \frac{n^2}{C^2 n^2} = \frac{1}{C^2}. \end{aligned}$$

You can choose $C \geq 10$, so $\Pr[T_n \geq n \log n + C \cdot n] \leq 1/C^2 \leq 0.01$.

Chernoff bounds:

(W9 - W10 starts here)

→ for a random var. X with $X \sim \text{Bin}(n, p)$, these are much more accurate than Markov or Chebyshev

Theorem: Let $X_1, \dots, X_n$ be independent Bernoulli-distributed random variables with $\Pr[X_i = 1] = p_i$

and $\Pr[X_i = 0] = 1 - p_i$. Then, for $X = \sum_{i=1}^n X_i$:

$$\text{i) } \Pr[X \geq (1 + \delta) \mathbb{E}[X]] \leq e^{-\frac{1}{2} \delta^2 \mathbb{E}[X]} \text{ for } 0 < \delta \leq 1$$

$$\text{ii) } \Pr[X \leq (1 - \delta) \mathbb{E}[X]] \leq e^{-\frac{1}{2} \delta^2 \mathbb{E}[X]} \text{ for } 0 < \delta \leq 1$$

$$\text{iii) } \Pr[X \geq t] \leq e^{-t} \quad \forall t \geq \mathbb{E}[X]$$

Coupon Collector: this is an important example to understand, similar algorithms may be featured on the exam

→ Consider n coupons, we receive one of them every round at random

→ Define the random variable $X :=$ amount of rounds until we have all n coupons

$\mathbb{E}[X] = ?$ → Consider n phases, where phase i is the round in which we own $i-1$ coupons,

including the round in which we obtain the i -th coupon. $X_i :=$ amount of rounds in phase i

$$\rightarrow \text{Then: } X = \sum_{i=1}^n X_i, X_i \sim \text{Geo}\left(\frac{n - (i-1)}{n}\right) \quad \leftarrow \Pr[\text{"round gives } i\text{-th coupon"}] = \frac{n - (i-1)}{n} \quad \leftarrow \begin{matrix} \text{new coupons} \\ \text{all coupons} \end{matrix}$$

↑ waiting for i -th success

↑ n -th harmonic number: $H_n = \sum_{i=1}^n \frac{1}{i} = \ln(n) + O(1)$

$$\rightarrow \mathbb{E}[X] \stackrel{\text{Lin. of } \mathbb{E}}{=} \sum_{i=1}^n \mathbb{E}[X_i] = \sum_{i=1}^n \frac{n}{n - (i-1)} = n \sum_{i=1}^n \frac{1}{n - (i-1)} = n \cdot H_n = O(n \log n)$$

↳ $\mathbb{E}[X_i] = \frac{1}{p}$ if $X \sim \text{Geo}(p)$

Bounding probabilities using **Markov**: We know $\mathbb{E}[X] = n \cdot H_n \leq n \cdot \ln n + n \leq 2 \cdot n \cdot \ln(n)$ (for $n \geq 3$)

Markov: $P[X \geq 100 \cdot \ln(n)] \leq \frac{\mathbb{E}[X]}{100 \cdot \ln(n)} = \frac{2 \cdot n \cdot \ln(n)}{100 \cdot \ln(n)} = \frac{n}{50}$ (prof's example on lecture notes was wrong)

Determining $\text{Var}[X]$:

$\rightarrow X = \sum_{i=1}^n X_i$, X_i = rounds in which we own $i-1$ unique coupons, $X_i \sim \text{Geo}(\frac{n-(i-1)}{n}) = \text{Geo}(1 - \frac{i-1}{n})$,

$X_1, \dots, X_n$ independent

$\rightarrow \text{Var}[X_i] = \frac{1-p_i}{p_i^2} \leq \frac{1}{p_i^2} = \left(\frac{n}{n-i+1}\right)^2$

$\rightarrow \text{Var}[X] = \text{Var}[\sum_{i=1}^n X_i] = \sum_{i=1}^n \text{Var}[X_i] \leq \sum_{i=1}^n \left(\frac{n}{n-i+1}\right)^2 = n^2 \sum_{i=1}^n \frac{1}{i^2} \leq n^2 \cdot \frac{\pi^2}{6}$

$\sum_{i=1}^{\infty} \frac{1}{i^2} = \frac{\pi^2}{6}$

$\rightarrow$ Example of **Chebyshev** for $t = n \cdot \sqrt{\ln n}$:

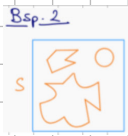
$\Rightarrow P[|X - \mathbb{E}[X]| \geq n \cdot \sqrt{\ln n}] \leq \frac{\text{Var}[X]}{t^2} \xrightarrow{n \rightarrow \infty} 0$

Target shooting algorithm (Chernoff):

$\rightarrow$ given: Sets S, U with $S \subseteq U$, goal: Find $\frac{|S|}{|U|}$

$\rightarrow$ Q1) is too slow, we want a probabilistic outcome

$\rightarrow$ Define ind. vars. $I_S : U \rightarrow \{0,1\}$ with $I_S(u) = 1 \Rightarrow u \in S$

$\rightarrow$ Example applications: 1. U = a population, S = infected people, 2.  $U = [0,1] \times [0,1]$, what area does S have?

Idea: Test if Elements are in S

Algorithm: ① Select $u_1, \dots, u_N$ from U randomly and independently with equal probability distributions

② Return $\frac{1}{N} \sum_{i=1}^N I_S(u_i)$

Intuitively: bigger subset chosen $\Rightarrow$ better approximation

Let ind. vars. $Y_i = I_S(u_i)$ for $i = 1, \dots, N$ and Y the algo.'s result $= \frac{1}{N} \cdot \sum_{i=1}^N Y_i$

$\rightarrow \mathbb{E}[Y] = \frac{1}{N} \sum_{i=1}^N \mathbb{E}[Y_i] = \frac{|S|}{|U|}$

Goal: $|Y - \frac{|S|}{|U|}| = |Y - \mathbb{E}[Y]| \rightarrow$ more precisely: for $\delta, \epsilon > 0$: $P[|Y - \frac{|S|}{|U|}| \leq \epsilon \frac{|S|}{|U|}] \geq 1 - \delta$

$\uparrow$ Error, we want it to be small with a high prob.

multiplicative error of $\leq (1 \pm \epsilon)$

Theorem: Let $\delta, \epsilon > 0$. If $N \geq \frac{1}{\epsilon^2} \cdot \frac{1}{\delta} \cdot \ln(\frac{2}{\delta})$, the returned variable Y of the target shooting algorithm is in the interval $[(1-\epsilon) \frac{|S|}{|U|}, (1+\epsilon) \frac{|S|}{|U|}]$ with probability $\geq 1 - \delta$.

Proof: $P[|Y - \frac{|S|}{|U|}| > \epsilon \frac{|S|}{|U|}] = P[|Y - \mathbb{E}[Y]| \geq \epsilon \cdot \mathbb{E}[Y]] = P[|NY - \mathbb{E}[NY]| \geq \epsilon \cdot \mathbb{E}[NY]]$

for which N does $P[|NY - \mathbb{E}[NY]| \geq \epsilon \cdot \mathbb{E}[NY]]$ hold.

$\rightarrow NY = Y_1 + \dots + Y_N$ = a sum of N independent Bernoulli variables $\rightarrow$ **Chernoff**:

$\rightarrow P[|NY - \mathbb{E}[NY]| > \epsilon \cdot \mathbb{E}[NY]] \leq 2 \cdot e^{-\frac{1}{2} \epsilon^2 \cdot \mathbb{E}[NY]} = 2 \cdot e^{-\frac{1}{2} \epsilon^2 N \cdot \frac{|S|}{|U|}} \leq \delta$ (for $N \geq \frac{1}{\epsilon^2} \cdot \frac{1}{\delta} \cdot \ln(\frac{2}{\delta})$)

Randomized Algorithms:

How deterministic algorithms work:

Input $I \rightarrow \text{Alg. } A \rightarrow \text{Output } A(I)$

$\rightarrow A(I)$ is always correct and

A always has the same asymptotic runtime

How probabilistic algorithms work:

Input $I \rightarrow \text{Alg. } A \rightarrow \text{Output } A(I, R)$

$\uparrow R$
Random variable

$\rightarrow$ The output depends on I and $R \rightarrow$ runtime or correctness can be variable and are not reproducible

- $\rightarrow$ In computational theory and our analysis, we assume physical random generation of the random bits used to create truly random numbers (Exercise: count how many times I said "random")
- $\rightarrow$ In practice, we use pseudo-random generators (deterministic) using a seed
- $\rightarrow$ Example: MD5 hashsums

"Üppige Auswahl" routine:

$\rightarrow$ given $a_1, \dots, a_n \in \{0, 1\}$, of which half equal 1, find an index i with $a_i = 1$

Trivial (and best) deterministic algorithm: for $i=1, \dots, n$: if $a_i=1$ return i

Theorem: every deterministic alg. needs at least runtime $\Omega(n)$.

Proof (I love this argumentation style!): For any given algorithm and n we create an instance:

$\rightarrow$ What index does the algorithm choose? Place a 0 there ...

$\rightarrow$ repeats until $n/2+1$, which places the alg. in $\Theta(n)$.

Las Vegas Algorithm (don't worry, we will discuss this):

repeat:

$i \leftarrow \text{Uniform}(\{1, \dots, n\})$
if $a_i=1$ return i

$$\begin{aligned} & \Pr[\text{"in the first } k \text{ repetitions, we have an index } i \text{ with } a_i=1"}] \\ &= \Pr[T > k] \quad (T \text{ is the random var. for the amount of repetitions}) \\ &= (1/2)^k \end{aligned}$$

Direct approach:

$$= \frac{1}{2} + \frac{1}{4} + \dots = 2$$

$$\mathbb{E}[T] = \sum_{n=1}^{\infty} \Pr[T \geq n] = \sum_{n=1}^{\infty} \Pr[T > n-1] = \sum_{n=1}^{\infty} (1/2)^{n-1}$$

$T \rightarrow$ amount of repetitions after first one fails

$$\text{Indirect approach: } \mathbb{E}[T] = \Pr[a_i=1] \cdot 1 + \Pr[a_i=0] \cdot (1 + \mathbb{E}[T]) \Rightarrow \mathbb{E}[T] = 1 + \mathbb{E}[T]/2 \Rightarrow \mathbb{E}[T]=2$$

$\rightarrow$ 2 repetitions: Algo runs in $O(1)$!

Monte Carlo and Las Vegas algorithms:

Monte Carlo algorithm: Correctness is a random variable, runtime is guaranteed

Las Vegas algorithm: Runtime is a random variable, Correctness is guaranteed

→ Alternatively, we can see them as algos. that sometimes return '???' instead of a result (if they don't finish in time)

Reducing error:

Las Vegas: Goal: Given the Las Vegas algorithm A with $\Pr[A(I) \text{ correct}] \geq \epsilon$, find a Las Vegas algorithm A_δ with $\Pr[A_\delta(I) \text{ correct}] \geq 1 - \delta$.

Idea: repeat A a sufficient amount of time → how often?

Theorem: Let A be a randomized algorithm that never returns a wrong answer, but '???' at times with $\Pr[A(I) \text{ correct}] \geq \epsilon$. Then, for all $\delta > 0$: Let A_δ be the algorithm that runs A until a value other than '???' is output, in which case this value is returned, or until '???' has been output $N = \lceil \epsilon^{-1} \ln \delta^{-1} \rceil$ times, in which case '???' is returned.

→ Reducing the probability of error only takes a constant amount of iterations → Example: $\epsilon = 0.25$

Monte Carlo: In general, it is not possible to reduce the probability of error

δ	$N = \lceil \epsilon^{-1} \ln \delta^{-1} \rceil$
0.1	10
0.01	19
0.001	28
0.0001	37
0.00001	47
0.000001	56

→ imagine a coinflip algorithm, you can't get a reliable answer

→ we can reduce the error prob. when there is a one-sided error or error prob. $< \frac{1}{2}$

→ one-sided error (*): $\Pr[A(I) = \text{YES}] = 1$ if I should return YES, $\Pr[A(I) = \text{NO}] \geq \epsilon$ if I should return NO

Theorem: Let A be a randomized algorithm that always returns YES or NO, following (*).

Then for all $\delta > 0$: Let A_δ be the algorithm that calls A until either NO is output and subsequently returned or YES is output $N = \lceil \epsilon^{-1} \ln \delta^{-1} \rceil$ times and subsequently returned.

Theorem: Let $\epsilon > 0$ and A be a randomized algorithm that always returns YES or NO with $\Pr[A(I) \text{ correct}] \geq \frac{1}{2} + \epsilon$. Then, for all $\delta > 0$: Let A_δ be the algorithm that calls A $N = \lceil 4\epsilon^{-2} \ln \delta^{-1} \rceil$ times and returns the majority of received answers. Then, $\Pr[A_\delta(I) \text{ correct}] \geq 1 - \delta$.

Proof of Las Vegas theorem: $N = \lceil \frac{1}{\epsilon} \cdot \ln \frac{1}{\delta} \rceil$. $\Pr[N \text{ times '???'}] \leq (1 - \epsilon)^N \leq e^{-\epsilon N} \leq e^{-\epsilon \cdot \frac{1}{\epsilon} \cdot \ln \frac{1}{\delta}} = e^{-\ln \frac{1}{\delta}} = \delta$.
 $\uparrow 1+x \leq e^x$

Proof of second MC theorem: we know $0 < \epsilon < \frac{1}{2}$ and define $X_i: \begin{cases} 1 & \text{if } i\text{-th call succeeds} \\ 0 & \text{else} \end{cases}$.
 $\left[X = \sum_{i=1}^N X_i \Rightarrow \mathbb{E}[X_i] = p \right] \quad p = \Pr[X_i = 1] = \Pr[A(I) \text{ corr.}] \geq \frac{1}{2} + \epsilon \Rightarrow \mathbb{E}[X] = N \cdot p = (\frac{1}{2} + \epsilon)N$
 $\uparrow \text{LoE}$

Also: $\frac{N}{2} \leq (1 - \epsilon)(\frac{1}{2} + \epsilon)N$. Now, compute $\Pr[A_\delta(I) \text{ correct}]$:

$\Pr[A_\delta(I) \text{ corr.}] \geq \Pr[X > \frac{N}{2}] = 1 - \Pr[X \leq \frac{N}{2}] \parallel \Pr[X \leq \frac{N}{2}] \leq \Pr[X \leq (1 - \epsilon) \mathbb{E}[X]] \leq e^{-\frac{1}{2}\epsilon^2 \mathbb{E}[X]} \leq e^{-\frac{1}{2}\epsilon^2 \cdot (\frac{1}{2} + \epsilon)N} \leq e^{-\ln \frac{1}{\delta}} = \delta$.
 $\uparrow \mathbb{E}[X] \geq \frac{N}{2} \geq \frac{1}{2\epsilon} \ln \frac{1}{\delta}$
 $\uparrow N = \lceil \frac{1}{\epsilon} \ln \frac{1}{\delta} \rceil$
 $\uparrow \text{we will use Chernoff}$

Randomized algorithms for optimization problems:

Theorem: Let $\epsilon > 0$ and A be a randomized algorithm for an optimization problem with $\Pr[A(I) \geq f(I)] \geq \epsilon$. 24

Then, for all $\epsilon > 0$: Let A_ϵ be the algorithm that calls A $N = \lceil \epsilon^{-1} \ln d^{-1} \rceil$ times and returns the best of the outputted answers, for which $\Pr[A_\epsilon \geq f(I)] \geq 1 - \epsilon$

Prime Number Tests:

Problem: Given $n \in \mathbb{N}$, decide if n is prime $\Rightarrow$ if it has no divider in $\{2, \dots, n-1\}$. The usual number size is 1000 bits, the prime number theorem allows for a high success rate ($\ln \frac{1}{1000 \ln 2} \approx \frac{1}{693}$)

Prime Number Theorem: $\pi(x) := |\{n \in \mathbb{N} \mid n \leq x, n \text{ prime}\}| \sim \frac{x}{\ln x}$

Naive Algorithm: for all $a \leq \sqrt{n}$, test if a divides $n \rightarrow O(\sqrt{n})$, but we want $O(\log n)$

Note: I will lightly touch up on group theory for the following algorithms, for a deeper review, check out the DM sheet

Euclidean Test: let $\gcd(m, n)$ be the GCD of $m, n \in \mathbb{Z}$ which can be computed using Euclid's Algorithm in $O((\log \max(m, n))^3)$. Trivially, $\gcd(a, n) > 1$ for $a \in [n-1] \Rightarrow n$ not prime

Euclidean Prime Test(n):

1. Choose $a \in [n-1]$ uniformly and randomly $\rightarrow$ if n prime $\rightarrow$ always correct
2. if $\gcd(a, n) = 1$ return 'prime' $\rightarrow$ if n not prime $\rightarrow$ wrong answer 'prime' with prob. $\frac{|\mathbb{Z}_n^*|}{n-1}$
3. else return 'not prime' $\rightarrow \mathbb{Z}_n^* = \{a \in [n-1] \mid \gcd(a, n) = 1\}$, $\varphi(n) := |\mathbb{Z}_n^*|$
 $\uparrow$ Euler's Totient

$\rightarrow n$ prime: $\mathbb{Z}_n^* = [n-1]$, $\varphi(n) = n-1$, $n = p^2$, p prime: $\varphi(n) = p(p-1) = n - \sqrt{n} \Rightarrow \frac{|\mathbb{Z}_n^*|}{n-1} \approx 1 - \frac{1}{\sqrt{n}}$

$\rightarrow \mathbb{Z}_n^*$ with multiplication generates a group of order $\varphi(n)$

Lagrange's Theorem: H subgroup of $G \Rightarrow |H|$ divides $|G|$

$\rightarrow \text{ord}(a) := \min \{i \in \mathbb{N} \mid a^i = 1\} \rightarrow$ order of subgroup $\langle a \rangle$ generated by $a := \{a^0 = 1, a^1, a^2, \dots\}$

$\rightarrow \text{ord}(a)$ divides $|G|$. Therefore, $a^{|G|} = a^{\text{ord}(a) \frac{|G|}{\text{ord}(a)}} = 1^{\frac{|G|}{\text{ord}(a)}} = 1$

$\rightarrow$ in $\mathbb{Z}_n^*$: $a^{\varphi(n)} = 1$ for all $a \in \mathbb{Z}_n^*$ ($\varphi(n) = |\mathbb{Z}_n^*|$)

$\rightarrow$ in $\mathbb{Z}_n^*$, n prime: $a^{n-1} = 1$ for all $a \in \mathbb{Z}_n^*$ ($\varphi(n) = n-1$)

Fermat's Little Theorem: if $n \in \mathbb{N}$ prime, for all numbers $a \in [n-1]$: $a^{n-1} \equiv_n 1$ ($a^{n-1} = 1$ in $\mathbb{Z}_n^*$)

Fermat Prime Number Test(n):

1. Uniformly and randomly choose $a \in [n-1]$
2. if $a^{n-1} \equiv_n 1$ return 'prime' $\rightarrow a^{n-1} \equiv_n 1$ can be sped up with binary exponentiation (Big Integer mod Pow is fast)
3. else return 'not prime' $\rightarrow n$ prime: always correct, n not prime: wrong answer 'prime' with prob. $\leq \frac{1}{2}$
 unless n is a Carmichael Number $\hookrightarrow$ pseudo-prime base

$\rightarrow$ repeat k times if 'prime' is output $\rightarrow$ bring error down $\frac{1}{2^k} \leq \frac{1}{2^k} \rightarrow$ Error: n not prime and $a^{n-1} \equiv_n 1$

$P_n := \{a \in [n-1] \mid a^{n-1} \equiv_n 1\} = \{a \in \mathbb{Z}_n^* \mid a^{n-1} \equiv_n 1\}$ (pseudo-prime bases)

$\overline{P_n}$

- n is a Carmichael number if it is not prime and $\phi_B = \mathbb{Z}_n^*$. Examples: $561 = 3 \cdot 11 \cdot 17$, $1105 = 5 \cdot 13 \cdot 17$
- ϕ_B subgroup of $\mathbb{Z}_n^*$ because $a^{n-1} \equiv_n 1 \wedge b^{n-1} \equiv_n 1 \Rightarrow (ab)^{n-1} \equiv_n 1$
- if $\phi_B \neq \mathbb{Z}_n^*$ (n not Carmichael), $|\phi_B|$ divides $|\mathbb{Z}_n^*|$ nontrivially and $|\phi_B| \leq \frac{\varphi(n)}{2} \leq \frac{n-1}{2}$

Miller-Rabin Certificate:

- for n prime, $(\mathbb{Z}_n, +, \cdot, 0, 1)$ is a field meaning that the equation $x^2 = 1$ has two solutions: 1 and $-1 (= n-1)$
- Let $n > 2$ odd. Write $n-1 = 2^k d$ with $k, d \in \mathbb{N}$, d odd. If n prime and $a \in [n-1]$:
 - $a^{2^k d} = a^{n-1} \equiv_n 1 \stackrel{\text{Fermat}}{\Rightarrow} a^{2^{k-1} d} \equiv_n 1 \text{ or } n-1 \Rightarrow$ if $a^{2^{k-1} d} \equiv_n 1$, $a^{2^{k-2} d} \equiv_n 1 \text{ or } n-1$
 - $\Rightarrow$ if $a^d \equiv_n 1$, $a^d \equiv_n 1 \text{ or } n-1 \Rightarrow a^d \equiv_n 1 \text{ or } \exists i \in \{0, \dots, k-1\} : a^{2^i d} \equiv_n n-1$.
- This is the certificate → Example: 561 (Carmichael): $2^{560} \equiv_{561} 1$, $2^{280} \equiv_{561} 1$, $2^{140} \equiv_{561} 67$
- $\Rightarrow 2$ is a M-R certificate showing that 561 is not prime
- $a \in \{2, 3\}$ is a M-R certificate for all $n < 1\,373\,653$

Algorithm (Miller-Rabin prime test (n) for $n > 1$, odd):

- Express $n-1 = 2^k d$ with $d, k \in \mathbb{N}$, d odd
 - Uniformly/randomly choose $a \in [n-1]$
 - if $a^d \equiv_n 1$ or $\exists i \in \{0, \dots, k-1\} : a^{2^i d} \equiv_n n-1$ then return 'prime'
 - else return 'not prime'
- n prime: always correct, n not prime: wrong answer 'prime' with prob. $\leq \frac{1}{2}$ (in reality $\leq \frac{1}{4}$)
 - I am purposefully omitting the proofs.

QuickSort:

QuickSort(A, l, r)

if $l < r$ then:

- $p \leftarrow \text{Uniform}(\{l, l+1, \dots, r\})$ // randomize pivot
- $t \leftarrow \text{Partition}(A, l, r, p)$ // "sort" $A[l], \dots, A[r]$ s.t. the elements left of $A[p]$ are $< A[p]$ and the elements right of $A[p]$ are $> A[p] \Rightarrow t$ is the new position of $A[p]$
- QuickSort($A, l, t-1$)
- QuickSort($A, t+1, r$)

Runtime analysis (exam relevant): Runtime $\rightarrow O(T_{l,r})$ with $T_{l,r}$ being a random variable counting the amount of comparisons made during QuickSort(A, l, r)

Theorem: $E[T_{l,r}] \leq 2(n-1) \ln(n) + O(n)$ ← constant

Proof: For $l \geq r$: $T_{l,r} = 0$, for $l < r$: $E[T_{l,r}] = \sum_{i=l}^r P[+ = i] \cdot (E[T_{l,i-1}] + E[T_{i+1,r}]) = \sum_{i=l}^r \frac{1}{r-l+1} \cdot (r-l + E[T_{l,i-1}] + E[T_{i+1,r}])$

Uniform prob.
 Partition makes $r-l$ comparisons
 recursive calls

Observation: $E[T_{l,r}]$ depends on the integer $r-l+1$

For $t_n = \begin{cases} 0 & \text{if } n \leq 1 \\ \frac{1}{n} \sum_{i=0}^{n-1} (n-1 + t_i + t_{n-i-1}) & \text{if } n \geq 2 \end{cases}$, $E[T_{l,r}] = t_{r-l+1}$ (proven by induction)

Goal: an upper bound for $E[T_1, \dots, n] = t_n \rightarrow \forall n \geq 3: n \cdot t_n = \sum_{i=0}^{n-1} (n-1+t_i+t_{n-i-1})$ (1)

$$(n-1) \cdot t_{n-1} = \sum_{i=0}^{n-2} (n-2+t_i+t_{n-i-2})$$
 (2)

$$(1)-(2): n \cdot t_n - (n-1)t_{n-1} = 2(n-1) + 2 \cdot t_{n-1}$$

$$\Rightarrow t_n = \frac{n+1}{n} t_{n-1} + \frac{2(n-1)}{n} \leq \frac{n+1}{n} t_{n-1} + 2 \quad (*)$$

Proposition: $\forall n \geq 2: t_n \leq 2 \sum_{i=3}^{n-1} \frac{n+1}{i}$

Proof by induction over n : Base Case $n=2 \rightarrow t_2 = 1 \leq 2 = 2 \sum_{i=3}^2 \frac{3}{i}$

$$n \geq 3: 2 \sum_{i=3}^{n+1} \frac{n+1}{i} = 2 \sum_{i=3}^n \frac{n+1}{i} + 2 \frac{n+1}{n+1} = \underbrace{2 \sum_{i=3}^n \frac{n+1}{i}}_{\geq t_{n-1}} + 2 \stackrel{(*)}{\leq} t_n$$

QuickSelect: given a sequence (array) $A[1], \dots, A[n]$ with pairwise distinct values, find the k -th smallest value

QuickSelect(A, l, r, k):

oh yeah, W11-W13 begins here

```

p ← Uniform( $\{l, l+1, \dots, r\}$ )
t ← Partition( $A, l, r, p$ ) // p is the t-th largest element
if t = l+k-1 then:
    return  $A[t]$ 
else if t > l+k-1 then:
    return QuickSelect( $A, l, t-1, k$ ) // target is to the left of t
else:
    return QuickSelect( $A, t+1, r, k$ ) // target is to the right of t
    
```

Expected runtime: $O(T)$, T is the random variable tracking the amount of comparisons (also exam relevant!)

$\rightarrow$ QuickSelect(A, l, r, k) causes N calls of Partition(A, l_i, r_i, p) for a random sequence $(l_0, r_0, k_0), \dots, (l_N, r_N, k_N)$

with $(l_0, r_0, k_0) = (l, r, k)$. Therefore, $(l_{i+1}, r_{i+1}) = (l_i, t-1)$ or $(t+1, r_i)$, where t is a uniformly distributed random variable

$$\rightarrow \text{Runtime of QuickSelect}(A, l, r, k) \rightarrow T = \sum_{i=1}^N (r_i - l_i)$$

$\leftarrow$ Partition(A, l_i, r_i, p) takes $r_i - l_i$ comparisons

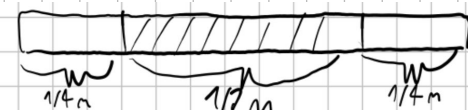
$\rightarrow$ We define N_j as amount of QuickSelect calls, for which $(3/4)^j n < r_i - l_i + 1 \leq (3/4)^{j-1} n$.

$$\text{Then, } T \leq \sum_{j=1}^{\infty} N_j \cdot \underbrace{(3/4)^{j-1} n}_{\text{LoE}} \stackrel{\text{LoE}}{\leq} E[T] \leq n \cdot \sum_{j=1}^{\infty} E[N_j] \cdot (3/4)^{j-1}$$

upper bound for comparisons of n partition calls

Observation: If the pivot lies in the middle $\frac{r-l+1}{2}$ elements,

the amount of considered $\underbrace{(r_i - l_i + 1)}_{(r_i - l_i + 1)}$ elements reduces to at least a factor of $3/4$



$\Rightarrow$ this has a probability of $1/2$ and is exactly what N_j measures $\Rightarrow E[N_j] \leq 2$ for all $j \geq 1$

$$\Rightarrow E[T] \leq 2n \sum_{j=1}^{\infty} (3/4)^{j-1} = 2n \sum_{j=0}^{\infty} (3/4)^j = 2n \cdot 4 = 8n$$

$\Rightarrow$ QuickSelect runs in linear time, which was our goal

Duplicates and HashMaps:

Problem: A dataset $D = (s_1, \dots, s_n)$ is a sequence of n elements. A tuple (i, j) with $1 \leq i \leq j \leq n$ is a duplicate

in D if $s_i = s_j$. Given a dataset D , find all duplicates.

Example: $D = (A, C, B, Z, C, B, C)$, set of duplicates in D : $\text{Dupl}(D) = \{(2,5), (2,7), (5,7), (3,6)\}$

Simpler algorithm: Sort $((s_1, 1), \dots, (s_n, n))$ by their first coordinate, iterate over the sorted sequence to find duplicates

Example: Indexing: $(A, 1) (C, 2) (B, 3) (Z, 4) (C, 5) (B, 6) (C, 7)$ Analysis: Sorting takes $O(n \log n)$ comparisons and memory access, iterating takes $O(n + |\text{Dupl}(D)|)$
 Sorting: $(A, 1) (B, 3) (B, 6) (C, 2) (C, 5) (C, 7) (Z, 4)$
 Duplicates: $(3, 6) (2, 5), (2, 7), (5, 7)$

Reason for Hashmaps: It is more efficient to compare hash values than large objects

Let the elements in D be from a universe U . We use a hash function $h: U \rightarrow [m]$ and require:

h is efficiently computable

h acts as a randomized function, meaning $\forall u \in U \forall i \in [m]: \Pr[h(u) = i] = \frac{1}{m}$

Important: Each $h(s_i)$ is randomized over $[m]$, but $s_i = s_j \Rightarrow h(s_i) = h(s_j)$, we compress by choosing m much smaller than U

Example solution:

Input: $A \quad C \quad B \quad Z \quad C \quad B \quad C$
 Hash and index: $(31, 1) \quad (27, 2) \quad (12, 3) \quad (12, 4) \quad (27, 5) \quad (12, 6) \quad (27, 7)$
 Sort: $(12, 3) \quad (12, 4) \quad (12, 6) \quad (27, 2) \quad (27, 5) \quad (27, 7) \quad (31, 1)$
 Duplicates: $(3, 6), (2, 5), (2, 7), (5, 7)$

Important: check for collisions while finding duplicates, such as $h(B) = h(Z)$

Collisions are the new, unwanted duplicates in a hashmap $\Rightarrow$ pairs (i, j) , $1 \leq i < j \leq n$ with $s_i \neq s_j$ and $h(s_i) = h(s_j)$

Probability of a collision: Let k_{ij} be the ind. var. of " (i, j) is a collision". Then:

$$\Pr[k_{ij} = 1] = \begin{cases} \frac{1}{m} & \text{if } s_i \neq s_j \\ 0 & \text{if } s_i = s_j \end{cases} \Rightarrow \mathbb{E}[k_{ij}] \leq \frac{1}{m} \forall i, j \Rightarrow \mathbb{E}[\text{\#collisions}] \leq \sum_{1 \leq i < j \leq n} \mathbb{E}[k_{ij}] \leq \sum_{1 \leq i < j \leq n} \frac{1}{m} \stackrel{\substack{= \frac{1}{m} \\ \text{pairs with } i < j \text{ are half of total}}}{\leq} \frac{1}{2} \cdot \frac{1}{m}$$

$\Rightarrow$ for $m = n$, $\mathbb{E}[\text{\#collisions}] < 1$, which only adds a constant factor to the runtime

Runtime: n hash function computations, $O(n \log n)$ to sort $\lceil 2 \log n \rceil$ -bit integers, $|\text{Dupl}(D)| + \text{\#collisions}$

$\Rightarrow$ same asymptotic runtime as simple algorithm but the comparisons themselves are less expensive for more complex objects

General Algorithms: (W10-W13 starts here)

Floyd's cycle finding alg. (Hare & Tortoise):

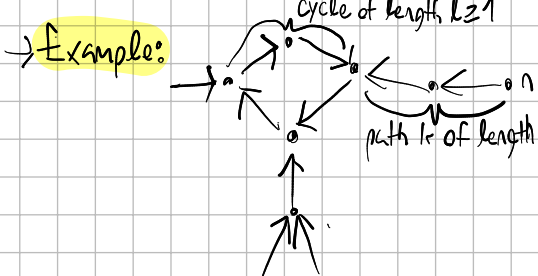
Given an array $a[1, \dots, n]$ with $a[i] \in \{1, \dots, n-1\}$ ($n-1$ unique values), find indices $i \neq j$ with $a[i] = a[j]$ in $O(n)$ time,

$O(1)$ space: doing one of each is easy; optimize space by remembering previous values in $O(n^2)$ time or use BucketSort for

$O(n)$ time and space.

Consider the directed graph $D = (V, A)$ with $V = [n]$, $A = \{(i, a[i]) \mid 1 \leq i \leq n\}$

how does D look? each vertex has one outgoing and one inging edge $\rightarrow$ the subgraph D_n consists of vertices reachable from n

Example:  $\rightarrow$ consider the sequence of vertices $x_0 = n, x_t := a[x_{t-1}] \forall t \geq 1$
 $\rightarrow$ Claim: $\exists t \leq n$ s.t. $x_t = x_{2t}$
 $\rightarrow$ Proof (optional): Let $r = \lceil \frac{n}{k} \rceil \cdot k - k \geq 0$. Then x_{k+r} is in the cycle C .
 $\Rightarrow k+r = \lceil \frac{n}{k} \rceil \cdot k$ is an integer multiple of k (length of C) $\Rightarrow x_{k+r} = x_{2(k+r)}$